

咕泡科技独家首发（非卖品）

Java 程序员

求职 突击 手册

@Tom 弹架构

目录

一、 简历模板	3
二、 面试突击押题（持续更新）	8
三、 一周技术提升清单	86
四、 薪资备查表	87
五、 职业路线图	88
六、 完整学习路线图	90

一、Java 各年限简历模板

1. 应届毕业生



教育经历

稻壳大学 (本科) 软件工程专业 2015.09-2019.07
主修课程: Coreldraw, dreamwear, Illustrator, Indesign、Java 基础代码、页面设计设计基础、网页代码基础、三大构成、广告设计与制作、版式设计等。

校园经历

稻壳大学 (本科) 策划部成员 2015.09-2019.07
1、策划杂志内容, 召开编辑会议, 分配工作任务, 撰写刊首文;
2、按照时间周期催稿, 审稿, 修改稿件, 将完稿交与研究会秘书长审阅;
3、与印刷厂商谈印刷事宜, 检查样本, 督促印刷厂按时按量完成印刷。

实习经历

阿帕奇网络科技有限公司 Java 工程师 2019.09-2020.04
1、负责产品前端开发工作, 利用 HTML (HTML5)、CSS (CSS3) 等前端技术将设计的产品转换成页面;
2、并利用 JS 和 jQuery 等框架实现页面的动态特效和交互式效果;
3、用 css3 实现动画效果, 让页面更加鲜活, 同时也能提高用户体验。

个人技能

熟练操作 Java、Coreldraw、dreamwear、C++ 语言、ios 开发 等设计软件。

所获证书

初级软件工程师证书、 计算机二级证书、
英语四级证书。

2. 1-3 年经验

	稻壳儿 求职意向: JAVA 工程师	📅 24 岁 📞 137 0000 0000 ✉ 123456@docer.com 📍 浙江杭州
教育背景	● 2012.9~2016.6 稻壳儿科技大学 计算机科学与技术 (本科) ➢ 主修课程: 计算机导论, C/C++ 语言编程, Java 语言编程, 算法与程序设计, 数据库、数据结构体、编译原理、操作系统、计算机组成原理、计算机网络、图形学、网络安全、数字电路、模拟电路、高等数学(微积分)、概率论与数理统计、线性代数、离散数学、图论等	
工作经历	● 2016.6~至今 杭州深蓝信息技术有限公司 JAVA 工程师 ➢ 参与公司项目研发工作、技术预研、技术难题攻关, 对开发软件质量把控 ➢ 参与项目的详细设计、负责业务功能模块的开发、单元测试、数据库设计 ➢ 控制系统及软件的维护及现场调试, 应用软件开发及系统性能和稳定性优化任务 ➢ 按要求提交软件开发过程中的标准化文档和操作手册, 维护、升级、优化内部系统, 负责编写公司基于 JAVA 的 web 项目后台代码, 参与重大研发项目	
专业技能	● 职业资格证: MCS D 微软国际认证、信息处理技术员、初级程序员、软件设计师 ● 语言: 普通话二级甲等, CET-6, 熟悉粤语、日语, 并能熟练听说读写 ● 编程能力: 熟悉关系型数据库产品 (MySQL、Oracle) 进行数据库编程, 熟悉 Apache、NginX、Tomcat、WildFly、Weblogic 等 Web 服务器和应用服务器	
荣誉奖励	● 2016 年度省级优秀毕业生 (获奖比例 1%) ● 《基于 SAAS 管理和数据库开发的深层次研究》获得浙江省优秀论文 ● 2015 年 12 月获浙江省 APP 设计大赛二等奖 (获奖比例 5%, 200 人参赛) ● 2015 年 4 月《多功能一站式医疗信息系统》获全国创新项目一等奖 ● 2014 年度获校“优秀共产党员”称号 ● 2014 年度校“优秀社团干部”, 并获得计算机系一等奖学金	
自我评价	● 精通 JAVA 语言和面向对象技术, 掌握 JSP,Servlet,JDBC 等 J2EE 相关技术 ● 熟悉 struts, spring, hibernate 等主流开发框架, 良好的项目掌控能力 ● 吃苦耐劳, 细心认真, 做事有条理, 大局观很强, 有奉献精神和实事求是态度 ● 熟悉 http 协议, html 语言和 JavaScript 脚本, 热爱学习, 极具创新思维 ● 熟悉 Oracle、DB2 等大型关系数据库, 有一定的数据库设计经验	

3. 3-5 年经验



曾慧华

求职意向: Java 软件工程师

个人信息

- 1996.12, 23 岁
- 最高学历: 本科
- 135-0000-0000
- zenghuihua123@qq.com

自我评价

两年以上 Java 软件工程师工作经验, 做过信用卡项目, 有多个银行项目经验, 抗压能力强;

拥有良好的代码习惯, 追求结构清晰, 命名规范, 逻辑性强, 代码冗余率低;

思维敏捷, 有较强的学习能力, 善于学习新的方法和技术, 能够独立分析问题和解决问题, 责任心强, 工作勤奋踏实, 富有激情, 有良好的沟通能力、团队协作精神和自我管理能力。

个人技能

1. 熟悉主流 Web 应用开发框架 (Spring MVC, Dubbo), 了解 Spring Boot, 熟悉主流应用服务器 (Tomcat, weblogic, Nginx 等);
2. 熟悉数据库设计和性能优化; 熟悉主流 RDBMS 和 NoSQL 数据库 (MySQL, Oracle, Redis 等), 以及数据库编程 (SQL, JDBC, iBatis 等);
3. 熟悉 Linux 系统常用命令;
4. 熟悉 angularjs, JavaScript, HTML, CSS;
5. 熟悉常用工程工具: Git/SVN, Jenkins, Maven/Npm, Eclipse/IntelliJ;
6. 熟悉设计模式, 熟练掌握面向对象编程和事件驱动编程风格;

工作经历

北京**技术有限公司

Java 开发工程师 2016.08 - 2019.06

1. 负责客户端 APP 或 APP 管理后台产品中服务器后端的工程设计, 架构设计, 接口文档编写以及开发工作;
2. 根据项目任务计划按时完成软件编码和单元测试工作;
3. 按照开发流程编写相应模块的设计文档;
4. 与产品经理、测试工程师、其他团队沟通合作, 保证产品研发工作的质量和进度;
5. 保质完成相关项目开发, 支撑生产环境的运营, 保证日程业务的正常运行;

项目经验

开发时间: 2018.02 - 至今

项目名称: “恒享生活”, 是恒丰银行信用卡官方客户端

开发环境: linux+oracle+eclipse

开发技术: spring mvc+fastdfs+hibernate+redis+nginx

项目背景: 恒丰银行把握移动互联网发展趋势, 克服线下网点数量较少的短板, 搭建对信用卡客户的门户 APP 平台, 成为各类业务广泛获客、精准营销的阵地与窗口。在初期该 APP 上仅实现信用卡的相关功能, 后续随着不断迭代增加新的业务模块。

项目介绍: ①APP 后台管理台, 有这些模块: APP 前端轮播图、APP 启动广告页面的管理、APP 协议管理、系统消息和热点新闻、黑白名单管理、版本管理、统计与分析模块、系统日志查询、

功能模块的维护、角色设置和用户设置、数据字典和系统参数的维护；②APP 后台项目模块：app 端通用业务的开发联调，包含 APP 里三个主要页面的轮播图；APP 每次启动的广告页面；版本更新；APP 中用户注册、卡片激活和绑定等协议；系统消息和热点资讯；数组字典和系统参数的查询。

责任描述：我一人整体负责 app 管理后台的后端代码编写。

项目成果：10 月 16 号成功在应用宝和小米应用商店上线。

开发时间：2016.06 - 2017.01。

项目名称：“稻壳”，稻壳银行 App。

开发环境：[linux+mysql+eclipse](#)。

开发技术：[spring mvc+dubbo+hibernate+redis](#)。

项目背景：为方便服务客户与推广银行业务，甘肃银行决定研发“我惠”甘肃银行 App。

项目介绍：是甘肃银行为广大客户提供的集金融服务、优惠权益为一体的手机客户端。在这里，你可以购买理财产品和黄金、积分兑换、享受客户经理专属服务。

责任描述：

1. 我负责产品中心、资源中心、交易中心、系统中心的部分交易的开发和维护以及接口文档的编写，这三个中心处于 [dubbo](#) 框架中“生产者”地位，所以在 [eclipse](#) 中是三个不同的子项目，单独开发。另外还有一个“消费者”的项目，接收 APP 的 HTTP 请求，把交易转到各个中心，完成处理请求。
2. 我负责理财产品购买、收益试算等相关模块，具体是 APP 后台开发我负责：对接理财系统，查询理财产品列表；查询理财产品详情；购买理财产品；收益试算；购买交易记录查询；APP 轮播图查询；APP 查询首页排版；APP 用到的所有协议查询。
3. 我负责管理台的后台开发：APP 轮播图的配置；APP 首页排版配置；APP 协议的配置。

项目成果：9 月 8 号在应用宝上线。

开发时间：2015.08 - 2016.06。

项目名称：自己公司的 OA 系统，包含 APP 应用和配套的管理台。

开发环境：[linux+mysql+eclipse](#)。

开发技术：前端技术 [angularjs+ionic+cordova](#)，后台技术：[spring mvc+hibernate+redis](#)。

项目背景：随着公司规模地不断发展壮大，移动内部办公需求加大，为提升管理水平，提高工作效率，公司开发了属于自己的 OA 管理系统和 APP。

项目介绍：该 OA 项目主要功能有：请假申请、出差申请、报销申请、查询请假、查询出差、查询报销、更改个人资料、年假计算。该 OA 项目前端是跨平台技术开发，一套代码可以分别打包成安卓和 [ios](#)，后台用的 [java](#)。

责任描述：项目前期 1.1 版本是两个后台人员在做，1.2 版本就是我一负责整个项目，包含 app 端后台开发和管理台后台开发。

项目成果：OA 系统自上线后一直运行良好。

教育背景

稻壳科技大学

工程管理

本科

2015.09 - 2017.12

稻壳建设职业技术学院

计算机应用技术

大专

2013.09 - 2016.06

4. 如何写好简历

写好简历的建议

简历是敲开企业大门的第一步！

好简历的标准：

美观：版式好看，字体好看

简明：内容简明扼要，最好 1 页纸

通顺：语句通顺，无病句，无错句

紧扣主题：始终抓住求职主题和求职意向

亮点突出：能够体现出围绕求职意向的个人优势

描述生动：简历内容清楚，具体化，容易理解

个人简历的三要

- 版面风格力求简洁、赏心悦目：简历的版面风格就像是一个人的外表，如果版面做得邋里邋遢或者过于花里胡哨，会直接影响招聘人员阅读简历的兴趣。还是应该尽量简洁，突出重点内容，适当的加以修饰，力求做到赏心悦目；
- 多用数据说话：特别是阐述求职者过往的工作经历时，尽量用具体的数据来量化你的工作业绩，忌讳使用“显著提高”、“主要贡献”等一些毫无用处的虚词；
- 因地制宜：根据不同的企业和岗位，有针对性的选择合适的简历模板和要表达的内容；

高通过率秘诀：

- 简历的命名非常重要，建议采用：“应聘+职位+姓名”的格式；
- 如果是直接发 HR 邮箱，可在邮箱中上传一页纸简历图片，方便 HR 查看；
- 建议在邮箱中上传 pdf 格式附件，更加有保障；
- 不建议直接上传 WORD，以免格式导致查看失败，如果一定要上传 word 先将简历修改成 03 或者 07 格式后缀，以免 HR 电脑 Office 版本过低，导致简历查看失败；
- 极简风格，无花哨装饰；格式有逻辑，标题行和描述行的高度要统一；
- 注重行文顺序，主体顺序是：基本信息-工作经历-项目经历-奖项-技能-自我评价；

二、Java 面试突击押题

1、Spring 中的 Bean 是线程安全的吗？

金三银四的招聘季到了，Spring 作为最热门的框架，在很多大厂面试中都会问到相关的问题。

前几天，就有好几个同学就问我，在面试中被问到这样一个问题。Spring 中的 Bean 是不是线程安全的。大家总觉得在面试过程差了一点意思。但是又说不上来是什么原因。这是因为，大家可能对 Spring 的本质还欠缺一些深度的思考。

今天，咱们不兜圈子不绕弯，上来直接说答案，大家关注点个赞，本视频跟大家彻底讲明白。

其实，Spring 中的 Bean 是否线程安全，其实跟 Spring 容器本身无关。Spring 框架中没有提供线程安全的策略，因此，Spring 容器中在的 Bean 本身也不具备线程安全的特性。咱们要透彻理解这个结论，我们首先要知道 Spring 中的 Bean 是从哪里来的。

1、Spring 中 Bean 从哪里来的？

在 Spring 容器中，除了很多 Spring 内置的 Bean 以外，其他的 Bean 都是我们自己通过 Spring 配置来声明的，然后，由 Spring 容器统一加载。我们在 Spring 声明配置中通常会配置以下内容，如：class（全类名）、id（也就是 Bean 的唯一标识）、scope（作用域）以及 lazy-init（是否延时加载）等。之后，Spring 容器根据配置内容使用对应的策略来创建 Bean 的实例。因此，Spring 容器中的 Bean 其实都是根据我们自己写的类来创建的实例。因此，Spring 中的 Bean 是否线程安全，跟 Spring 容器无关，只是交由 Spring 容器托管而已。

那么，在 Spring 容器中，什么样的 Bean 会存在线程安全问题呢？回答，这个问题之前我们得先回顾一下 Spring Bean 的作用域。在 Spring 定义的作用域中，其中有 prototype（多例 Bean）和 singleton（单例 Bean）。那么，定义为 prototype 的 Bean，是在每次 `getBean` 的时候都会创建一个新的对象。定义为 singleton 的 Bean，在 Spring 容器中只会存在一个全局共享的实例。基于对以上 Spring Bean 作用域的理解，下面，我们来分析一下在 Spring 容器中，什么样的 Bean 会存在线程安全问题。

2、Spring 中什么样的 Bean 存在线程安全问题？

我们已经知道，多例 Bean 每次都会新创建新实例，也就是说线程之间不存在 Bean 共享的问题。因此，多例 Bean 是不存在线程安全问题的。

而单例 Bean 是所有线程共享一个实例，因此，就可能会存在线程安全问题。但是单例 Bean 又分为无状态 Bean 和有状态 Bean。在多线程操作中只会对 Bean 的成员变量进行查询操作，不会修改成员变量的值，这样的 Bean 称之为无状态 Bean。所以，可想而知，无状态的单例 Bean 是不存在线程安全问题的。但是，在多线程操作中如果需要对 Bean 中的成员变量进行数据更新操作，这样的 Bean 称之为有状态 Bean，所以，有状态的单例 Bean 就可能存在线程安全问题。

所以，最终我们得出结论，在 Spring 中，只有有状态的单例 Bean 才会存在线程安全问题。我们在使用 Spring 的过程中，经常会使用到有状态的单例 Bean，如果真正遇到了线程安全问题，我们又该如何处理呢？

3、如何处理 Spring Bean 的线程安全问题？

处理有状态单例 Bean 的线程安全问题有以下三种方法：

- 1、将 Bean 的作用域由 “singleton” 单例 改为 “prototype” 多例。
- 2、在 Bean 对象中避免定义可变的成员变量，当然，这样做不太现实，就当我没说。
- 3、在类中定义 `ThreadLocal` 的成员变量，并将需要的可变成员变量保存在 `ThreadLocal` 中，`ThreadLocal` 本身就具备线程隔离的特性，这就相当于为每个线程提供了一个独立的变量副本，每个线程只需要操作自己的线程副本变量，从而解决线程安全问题。

都已经看到这里了，相信大家应该已经知道了 Spring 中的 Bean 是否线程安全以及如何处理 Bean 的线程安全问题。

2. 谈谈你对 Spring Bean 的理解

【引言】

大家好，我是被编程耽误的文艺 Tom。

前几天，有位同学向我反馈，说在面试中间到这样一个面试题：谈谈你对 Spring Bean 的理解。今天咱们就针对这样一个面试题，给大家做一个详细的介绍。我一共分三段来介绍，首先，介绍什么是 Spring Bean？然后，定义 Spring Bean 有哪些方式？，最后，给大家介绍 Spring 容器是如何加载 Bean 的？

咱们先来看什么是 Spring Bean？

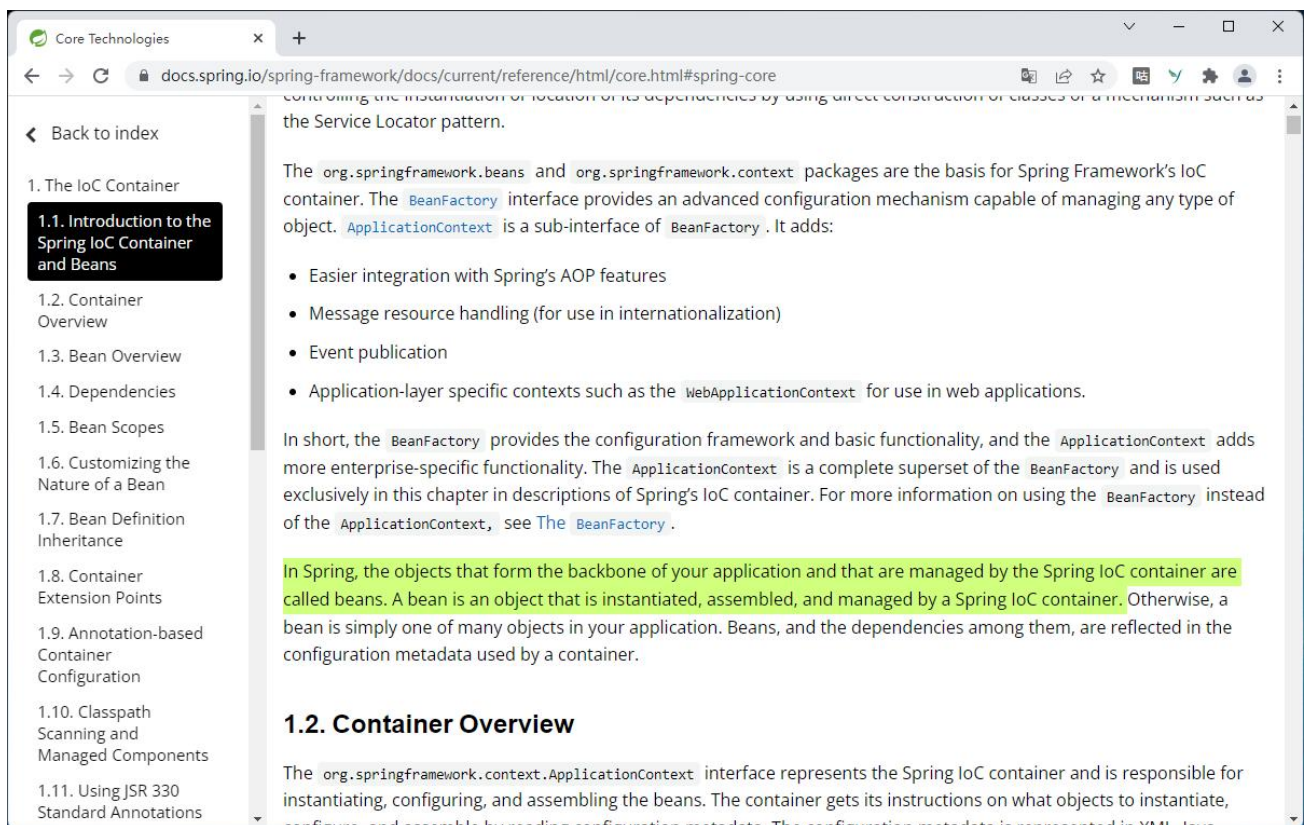
1、什么是 Spring Bean？

Spring Bean 是 Spring 中最基本的组成单元，Spring 官方文档对 Bean 的解释是这样的：

In Spring, the objects that form the backbone of your application and that are managed by the Spring IoC container are called beans. A bean is an object that is instantiated, assembled, and managed by a Spring IoC container.

翻译过来就是：

在 Spring 中，构成应用程序主干并由 Spring IoC 容器管理的对象称为 Bean。Bean 是一个由 Spring IoC 容器实例化、组装和管理的对象。



从官方定义中，我们可以提取出以下信息：

- 1、Bean 是对象，一个或者多个不限定。
- 2、Bean 托管在 Spring 中一个叫 IoC 的容器中。
- 3、我们的程序是由一个个 Bean 构成的。

Spring 是通过声明式配置的方式来定义 Bean 的，所有创建 Bean 需要的前置依赖或者参数都是通过配置文件先声明，Spring 启动以后会解析这些声明好的配置内容。那么，我们该如何去定义 Spring 中的 Bean 呢？

2、定义 Spring Bean 有哪些方式？

一般来说，Spring Bean 的定义配置有三种方式：

第一种：基于 XML 的方式配置

这种配置方式主要适用于以下两类场景：

- 1、Bean 实现类来自第三方类库、比如 DataSource 等
- 2、需要定义命名空间的配置，如：context、aop、mvc 等。

举个例子，来看一段代码

```
<beans>
  <import resource="resource1.xml" /> //导入其他配置文件 Bean 的定义
  <import resource="resource2.xml" />
  <bean id="userService" class="com.gupaoedu.vip.UserService" init-method="init"
destory-method="destory">
    </bean>
  <bean id="message" class="java.lang.String">
    <constructor-arg index="0" value="test"></constructor-arg>
```

```
</bean>
</beans>
```

这段代码是标准的 Spring 配置内容，咱们从上往下来看，首先导入一个叫做 resource1.xml 的标准配置文件，然后，导入了第二个标准的配置文件 resource2.xml，接着，定义了一个叫做 userService 的 Bean，对应的类是 com.gupaoedu.vip.UserService，并且声明了在 userService 实例化之后要调用 init() 方法，在 userService 销毁之后调用 destroy() 方法。

第二种：基于注解扫描的方式配置

这种配置方式主要适用于：在开发中需要引用的类，如 Controller、Service、Dao 等。两种配置方法：

```
<context:component-scan base-package="com.gupaoedu.vip">
    <context:include-filter type="regex" expression="com.gupaoedu.vip.mall.*"/> //包含的目标类
    <context:exclude-filter type="aspectj"
expression="com.gupaoedu.vip.mall.*Controller+"/> //排除的目标类
</context:component-scan>
```

在这段配置中，context:component-scan 相当于使用了 Spring 内置的扫描注解的组件 @ComponentScan，声明了需要扫描的基础包路径 com.gupaoedu.vip，把所有 com.gupaoedu.vip.mall 下面的子包全部纳入扫描范围。并且排除了 com.gupaoedu.vip.mall 包下面所有以 Controller 结尾或者包含 Controller 结尾的类。

使用 Spring 容器管理组件的 beanName 规则，默认是类名首字母变小写，可以自己修改。Spring 主要提供了 4 种内置的注解用来声明 Bean。它们分别是@Controller，声明为控制层组件的 Bean，@Service，声明为业务逻辑层组件的 Bean，@Repository，声明为数据访问层组件的 Bean，当对组件的层次难以定位的时候使用@Component 注解来声明。

Spring 提供了四个注解，这些注解的作用与上面的 XML 定义 bean 效果一致，在于将组件交给 Spring 容器管理。组件的名称默认是类名（首字母变小写），可以自己修改：

第三种：基于 Java 类的配置

这种配置方式主要适用于以下两类场景：

- 1、需要通过代码控制对象创建逻辑的场景；
- 2、实现零配置，消除 XML 配置文件的场景。

同样还是举个例子，来看一段代码：

```
@Configuration
public class BeansConfiguration {

    @Bean
    public Student student() {

        Student student = new Student();
        student.setName("张三");
        student.setTeacher(teacher());
        return student;
    }
}
```

```
}

@Bean
public Teacher teacher() {

    Teacher teacher = new Teacher();
    teacher.setName("Tom");
    return teacher;

}
}
```

通过分析这段代码，我可以了解到使用基于类的配置需要以下步骤：

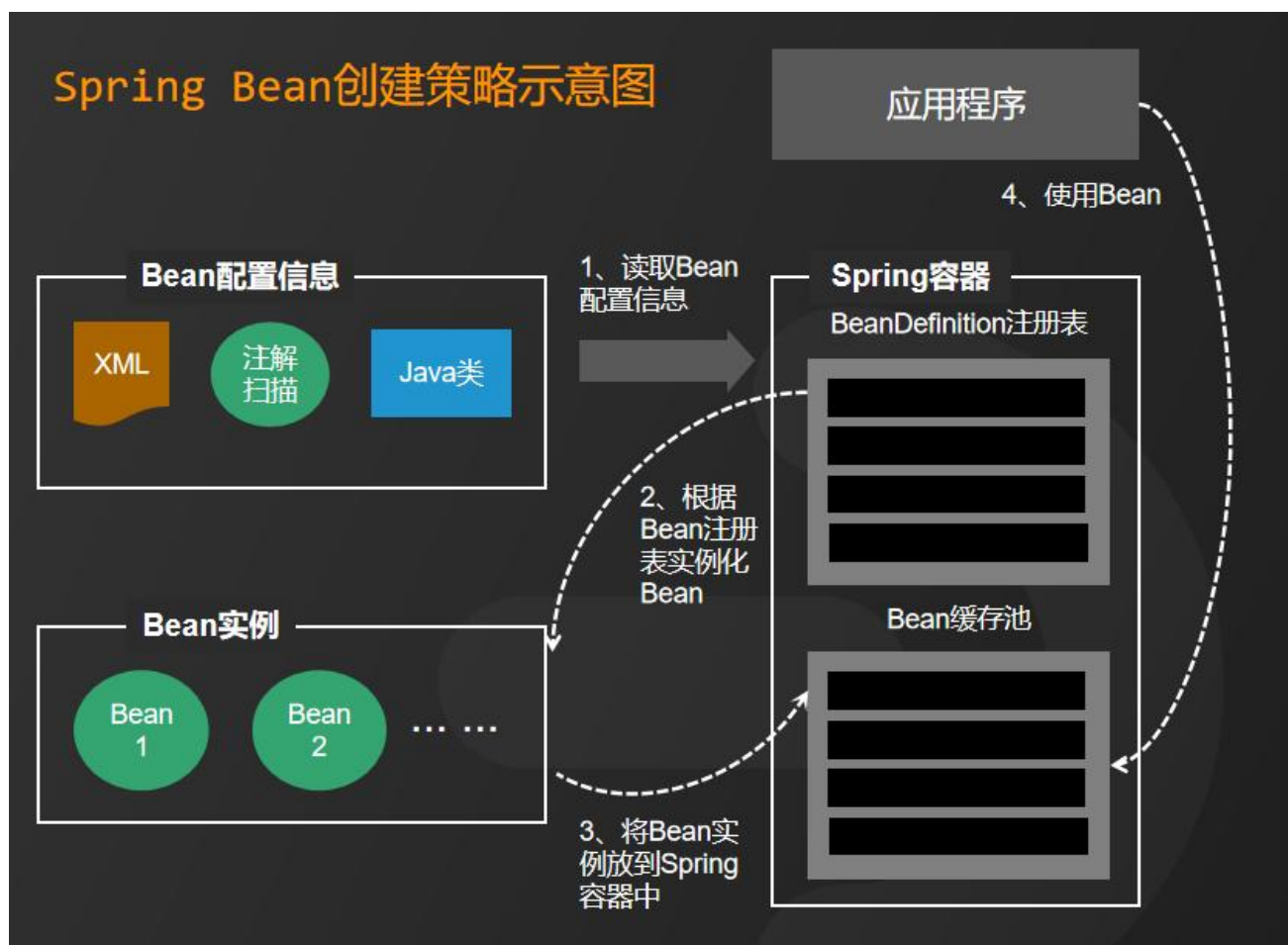
- 1、首先，在 BeansConfiguration 类上配置@Configuration 注解，表示将 BeansConfiguration 类将定义 Bean 的元数据
- 2、在方法上使用@Bean 注解，方法名默认就是 Bean 的名称，方法返回值就是 Bean 的实例。
- 3、通过 AnnotationConfigApplicationContext 或子类来启动 Spring 容器，从而加载这些已经声明好的注解配置。

最后，总结一下，定义 Spring Bean 三种方式，分别是：基于 XML 的方式配置、基于注解扫描的方式配置、基于元数据类的配置。

那么 Bean 的配置定义好以后，Spring 又是如何加载这些 Bean 配置并创建 Bean 实例呢？

3、Spring 容器如何加载 Bean？

Spring 解析这些声明好的配置内容，将这些配置内容都转化为 BeanDefinition 对象，BeanDefinition 中几乎保存了配置文件中声明的所有内容，再将 BeanDefinition 存到一个叫做 beanDefinitionMap 中。以 beanName 作为 Key，以 BeanDefinition 对象作为 Value。之后 Spring 容器，根据 beanName 找到对应的 BeanDefinition，再去选择具体的创建策略。而 Spring 具体的创建策略如图所示，大家想要高清无码图的可以在评论区留言。



关于 Spring Bean 的面试题解析就到这里，相信大家也已经理解。如果想看 Spring BeanDefinition 定义的小伙伴，可以关注我后续的更新内容。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，也可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

3. Spring Bean 的定义包含哪些内容？

【引言】

大家好，我是被编程耽误的文艺 Tom。

前面我发了一个关于 Spring Bean 的视频。在这个视频中，我简单提到了 Spring Bean 的定义。其中，有几位同学就私信我，说老师能不能拍一期关于 Spring Bean 定义的详细介绍，今天我就来满足大家的要求。关于 Spring Bean 的定义我一共分为三部分来介绍，首先，介绍 Spring Bean 声明式配置内容；然后，介绍 BeanDefinition 与配置文件的关系；最后，介绍 Spring 如何解析配置文件？

咱们先来看 Spring Bean 声明式配置内容有哪些？

1、Spring Bean 声明式配置内容

关于 Spring Bean 的配置内容非常多，我主要列举九个关键的配置属性，比如：class、scope、lazy-init、depends-on、name、constructor-arg、properties、init-method、destroy-method 等。

这些属性都是要在 Spring 配置文件中声明的内容。在 Spring 容器启动后，这些配置内容都会映射到一个叫做 `BeanDefinition` 的对象中。然后，所有的 `BeanDefinition` 对象都会保存到一个叫做 `beanDefinitionMap` 的容器中，这个容器是 `Map` 类型，以 Bean 的唯一标识作为 key，以 `BeanDefinition` 对象实例作为值。这样 Spring 容器创建 Bean 时，就不需要再次读取和解析配置文件，只需要根据 Bean 的唯一标识，去 `beanDefinitionMap` 中取到对应的 `BeanDefinition` 对象即可。

那么，接下来我们看一下 `BeanDefinition` 是如何定义的。

2、BeanDefinition 与配置文件的关系

我们可以对照源码来看，`BeanDefinition` 的基础实现类 `AbstractBeanDefinition` 类，这个类下面的所有属性都能够和声明配置文件中的内容一一对应上，来看代码：

```
public AbstractBeanDefinition implements BeanDefinition {  
  
    ...  
  
    @Nullable  
    private volatile Object beanClass;  
  
    @Nullable  
    private String scope = SCOPE_DEFAULT;  
  
    private boolean lazyInit = false;  
  
    @Nullable  
    private String[] dependsOn;  
  
    @Nullable  
    private String factoryBeanName;  
  
    @Nullable  
    private ConstructorArgumentValues constructorArgumentValues;  
  
    @Nullable  
    private MutablePropertyValues propertyValues;  
  
    @Nullable
```

```
private String initMethodName;

@Nullable
private String destroyMethodName;

...
}
```

我们可以看到,BeanDefinition中定义的属性和声明式的配置内容从命名上看比较类似。本期视频中,我重点介绍 5 个:

- 1、beanClass 对应的配置是 class, 这个属性为必填项, 用于指向一个具体存在的 Java 类, Spring 容器创建的 Bean 就是这个 Java 类的实例。
- 2、lazyInit 对应的配置是 lazy-init, 用于指定 Bean 实例是否延时加载, 我们能清楚地看到默认值是 false。也就是说容器启动时就会创建 Bean 对应的实例, 如果设置为 true, 则只有在首次获取 Bean 的实例时才创建。
- 3、dependsOn 对应的配置是 depends-on, 用于定义 Bean 实例化的依赖关系。在 Spring 容器对 Bean 的实例初始化之前, 有可能存在其他依赖, 这需要需要保证其所依赖的 Bean 需要提前实例化, depends-on 可以用来定义 Bean 的依赖顺序。在 BeanDefinition 中属性定义的数据类型是字符串数组, 也就是说可以同时定义多个依赖对象。
- 4、factoryBeanName 对应的配置就是 name, 这个属性用于定义 Bean 的唯一标识, 且不能以大写字母开头。在 XML 配置中, 使用 id 或 name 属性来指定。如果没有设值, Spring 默认使用类名首字母小写作为唯一标识。
- 5、constructorArgumentValues 对应的配置是 constructor-arg, 它其实也是一个数组。如果 Java 类中定义了有参构造方法, 则可以使用此属性给有参构造方法注入参数值。如果没有默认的无参构造方法, 那么, 这个属性必填。

其他的属性我相信小伙伴根据属性名称也能够自己一一对应上。我呢, 也给大家整理成了一个表格, 有需要完整表格的小伙伴可以在评论区留言, 可以发给大家。

Spring Bean 声明式配置和 BeanDefinition 属性定义对照表

序号	配置	对应属性	说明
1	class	beanClass	这个属性为必填项，用于指向一个具体存在的Java类，Spring容器创建的Bean就是这个Java类的实例
2	scope	scope	用于定义作用域，默认值为singleton，单例，一共有singleton、prototype、request、session、globalSession 五个值可选，不同的作用域将由Spring选择特定的策略创建Bean的实例。
3	lazy-init	lazyInit	用于指定Bean实例是否延时加载。默认为false，也就是说容器启动时就会创建Bean对应的实例，如果设置为true，则只有在首次获取Bean的实例时才创建。
4	depends-on	dependsOn	用于定义Bean实例化的依赖关系。在Spring容器对Bean的实例初始化之前，有可能存在其他依赖，这需要需要保证其所依赖的Bean需要提前实例化，depends-on可以用来定义Bean的依赖顺序。
5	name	factoryBeanName	这个属性用于定义Bean的唯一标识，且不能以大写字母开头。在XML配置中，使用id或name属性来指定。如果没有设置，Spring默认使用类名首字母小写作为唯一标识。
6	constructor-arg	constructorArgumentValues	如果Java类中定义了有参构造方法，则可以使用此属性给有参构造方法注入参数值。如果没有默认的无参构造方法，那么，这个属性必填。
7	properties	propertyValues	用于给Bean的指定属性赋默认值。
8	init-method	initMethodName	用于指定在Bean完成初始化之后需要调用的回调方法。当Bean的所有必需的属性被容器设置之后，意味着Bean初始化完成。
9	destroy-method	destroyMethodName	用于指定包含该Bean的实例被销毁时，需要调用的回调方法。

对照源码看完之后，大家应该非常清楚 Spring Bean 定义的关键内容包含哪些属性了。那么，Spring 又是如何解析这些配置文件变成 BeanDefinition 对象的呢？

3、Spring 如何解析配置文件？

Spring 容器启动之后，会调用 BeanDefinitionReader 工具类的 loadBeanDefinitions() 方法，启动对配置文件的加载和解析。BeanDefinitionReader 的主要作用是读取 Spring 配置文件中的内容，将其转换为 BeanDefinition 对象。而 BeanDefinitionReader 又有非常多的实现类，每种类型的配置具体解析的过程又不一样，比如 XmlBeanDefinitionReader，用于读取 XML 文件并解析为 BeanDefinition 对象。PropertiesBeanDefinitionReader，用于读取属性文件，将 Resource，Property 等解析为 BeanDefinition 对象。GroovyBeanDefinitionReader，用于读取 Groovy 语言定义的 Bean，将它们解析为 BeanDefinition 对象。

以上就是关于 Spring Bean 定义的全部解析。我是被编程耽误的文艺 Tom，如果大家还有其他疑问，可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

4. 为什么 Spring 中每个 Bean 都要定义作用域？

【引言】

大家好，我是被编程耽误的文艺 Tom。

前面的视频中都有提到过 Spring Bean 的作用域。本期视频呢，我针对 Spring Bean 作用域做一个详细的解答。关于 Spring Bean 的作用域，我一共分为两个部分来介绍。首先，介绍 Spring Bean 作用域的定义，然后，介绍 Spring 为什么要定义作用域？咱们先来看 Spring Bean 作用域的定义有哪些？

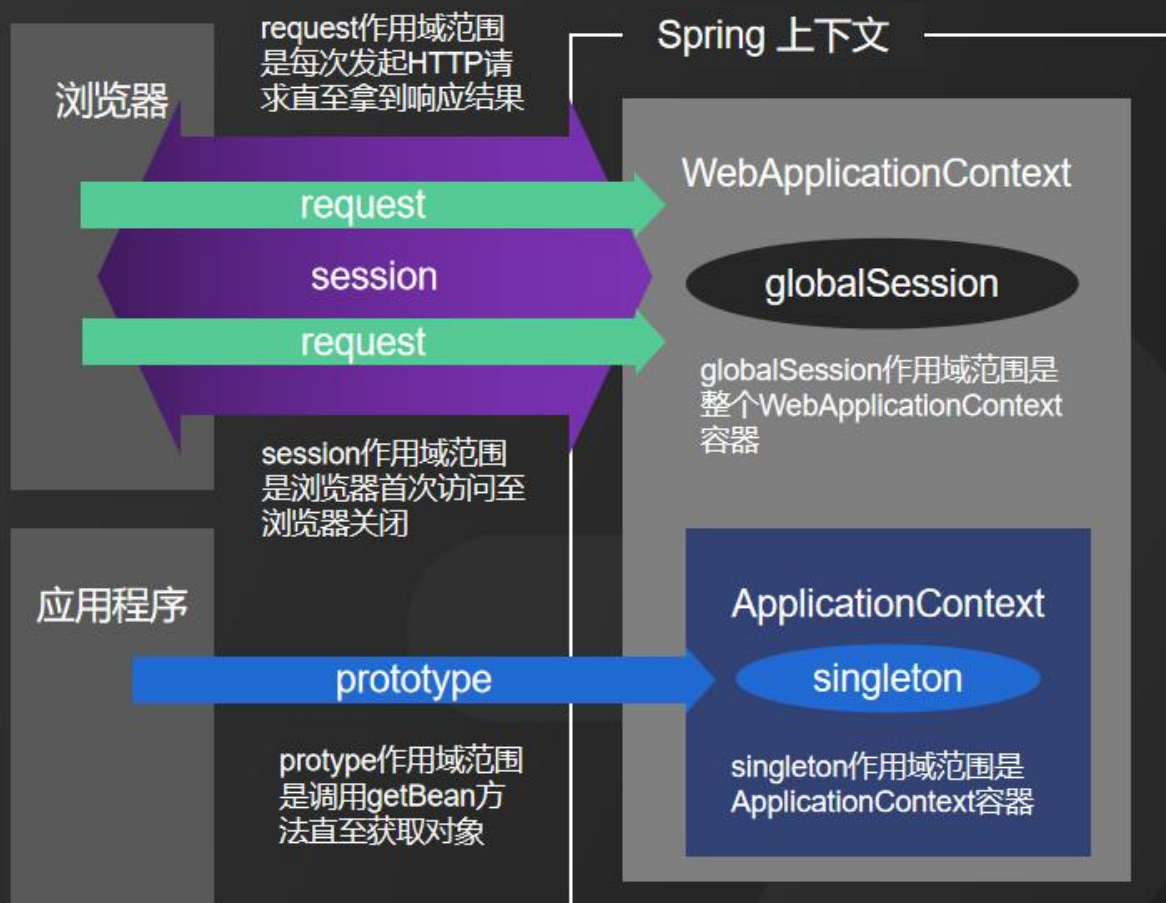
1、Spring Bean 作用域的定义

在 Spring 配置中，我们可以通过 scope 属性来定义 Spring Bean 的作用域，可以接受 5 个内建的值，分别代表 5 种作用域类型，下面给大家详细总结一下：

- 1、singleton，用来定义一个 Bean 为单例，也就是说在 Spring IoC 容器中仅有唯一的一个实例对象，Spring 中的 Bean 默认都是单例的。它的作用域范围是 ApplicationContext 容器
- 2、prototype，用来定义一个 Bean 为多例，也就是说在每次请求获取 Bean 的时候都会重新创建实例，因此每次获取到的实例对象都是不同的。它的作用域范围是调用 getBean 方法直至获取对象。
- 3、request，用来定义一个作用范围仅在 request 中的 Bean，也就是说在每次 HTTP 请求时会创建一个实例，该实例仅在当前 Request 中有效。它的作用域范围是每次发起 HTTP 请求直至拿到响应结果。
- 4、session，用来定义一个作用范围仅在 session 中的 Bean，也就是说在每次 HTTP 请求时会创建一个实例，该实例仅在当前 HTTP Session 中有效。它的作用域范围是浏览器首次访问至浏览器关闭。
- 5、globalSession，用来定义一个作用范围仅在其中的 Bean。这种方式仅用于应用环境，也就是说该实例仅存在于 WebApplicationContext 环境中。它的作用域范围是整个 WebApplicationContext 容器。

第一个 singleton 和第二个 prototype 是比较常用的。其他三种仅适用于 Web 应用环境中，咱们也无须关心用什么样的框架，只需要符合 J2EE 规范即可生效。

Spring Bean作用域示意图



各种作用域范围大小对比



这一张图呢，是表示各种作用域范围大小对比，其中 prototype 大于 request 大于 session 大于 globalSession 大于 singleton。大家可以私信我获取高清图，下载下来慢慢看，帮助大家更好地理解作用域范围。

2、Spring 为什么要定义作用域？

定义 Bean 的作用域，相当于用户可以通过配置的方式限制 Spring Bean 的使用范围，以起到保护 Bean 安全的作用。就好比孙悟空外出打妖怪前，给唐僧画了一个圈。唐僧只有待在圈里才能保证安全，出圈就可能会遇到危险。这样，唐僧访问不到圈外的资源，圈外的资源也无法触达到唐僧，以此形成一个安全的隔离区。



在日常开发中，我们可以根据业务需要，选择定义不同的作用域，以保护 Bean 的使用安全。

关于 Spring Bean 的作用域解析就到这里。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，也可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。
关注我，面试不再难！

5. Spring Bean 的生命周期全过程

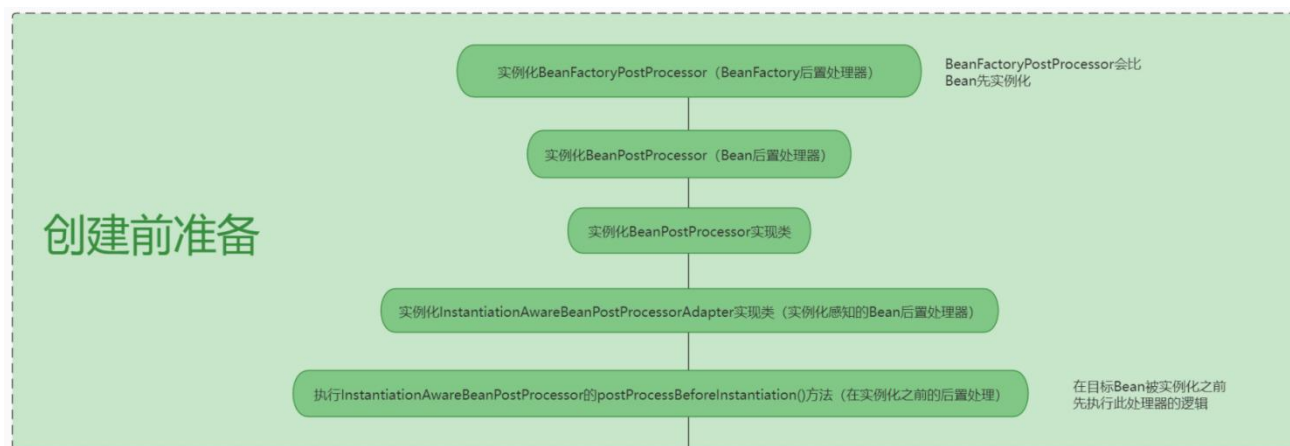
大家好，我是被编程耽误的文艺 Tom。今天能给大家介绍一下 Spring Bean 生命周期全过程，这道题呢也是大厂高频面试题。接下来我给大家做一个详细的分析和解答。

Spring 生命周期全过程大致分为五个阶段：创建前准备阶段、创建实例阶段、依赖注入阶段、容器缓存阶段和销毁实例阶段。

这张图呢展示了 Spring Bean 生命周期完整流程，其中对每个阶段的具体操作做了详细介绍。下面我们来分析一下每个阶段的细节。

先来看第一阶段。

1、创建前准备阶段



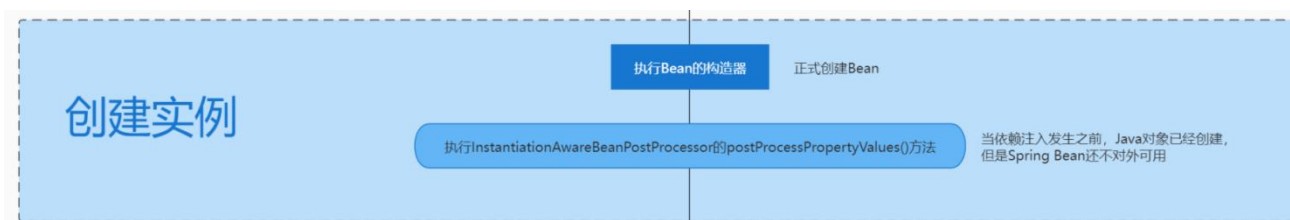
这个阶段主要是在开始 Bean 加载之前，从 Spring 上下文和相关配置中解析并查找 Bean 有关的配置内容，

比如`init-method`-容器在初始化 bean 时调用的方法、`destroy-method`，容器在销毁 Bean 时调用的方法。

以及，BeanFactoryPostProcessor 这类的 bean 加载过程中的前置和后置处理。

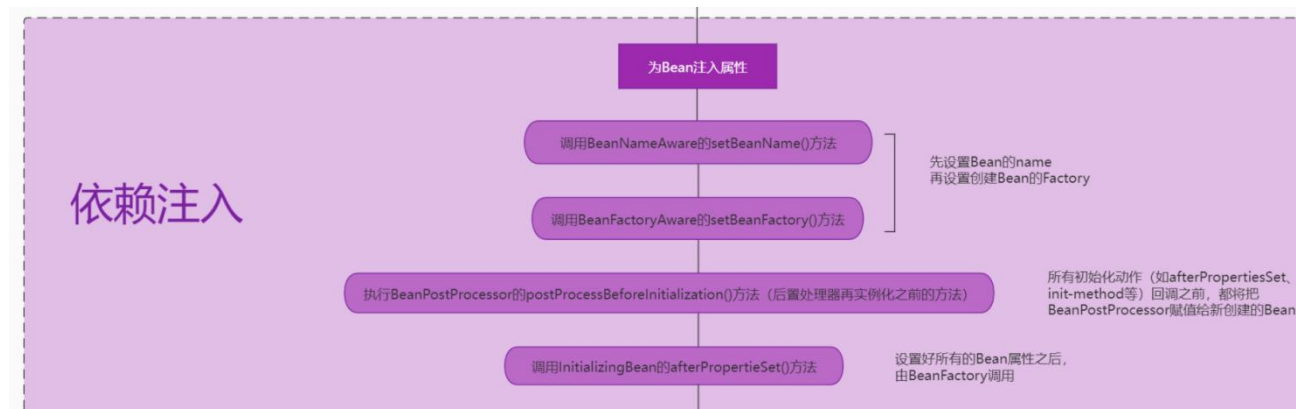
这些类或者配置其实是 Spring 提供给开发者，用来实现 Bean 加载过程中的扩展机制，在很多和 Spring 集成的中间件经常使用，比如 Dubbo。

2、创建实例阶段



这个阶段主要是通过反射来创建 Bean 的实例对象，并且扫描和解析 Bean 声明的一些属性。

3、依赖注入阶段



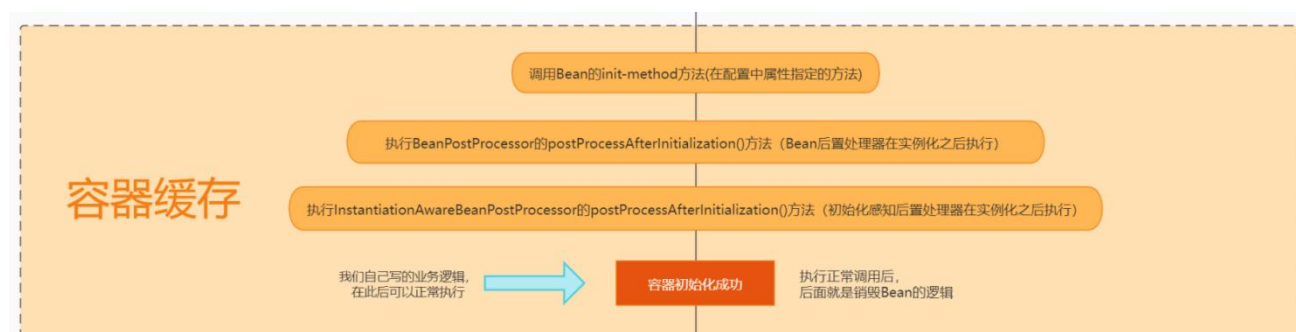
在这个阶段，会检测被实例化的 Bean 是否存在其他依赖，如果存在其他依赖，就需要对这些被依赖 Bean 进行注入。比如通过读取

`@Autowired`、`@Setter` 等依赖注入的配置。

在这个阶段还会触发一些扩展的调用，比如常见的扩展类：BeanPostProcessors（用来实现 Bean 初始化前后的回调）、

InitializingBean 类（这个类有一个 `afterPropertiesSet()` 方法，给属性赋值）、还有 BeanFactoryAware 等等。

4、容器缓存阶段



容器缓存阶段主要是把 Bean 保存到 IoC 容器中缓存起来，到了这个阶段，Bean 就可以被开发者使用了。

这个阶段涉及到的操作，常见的有，`init-method` 这个属性配置的方法，会在这个阶段调用。

在比如 BeanPostProcessors 方法中的后置处理器方法如：`postProcessAfterInitialization`，也是在这个阶段触发的。

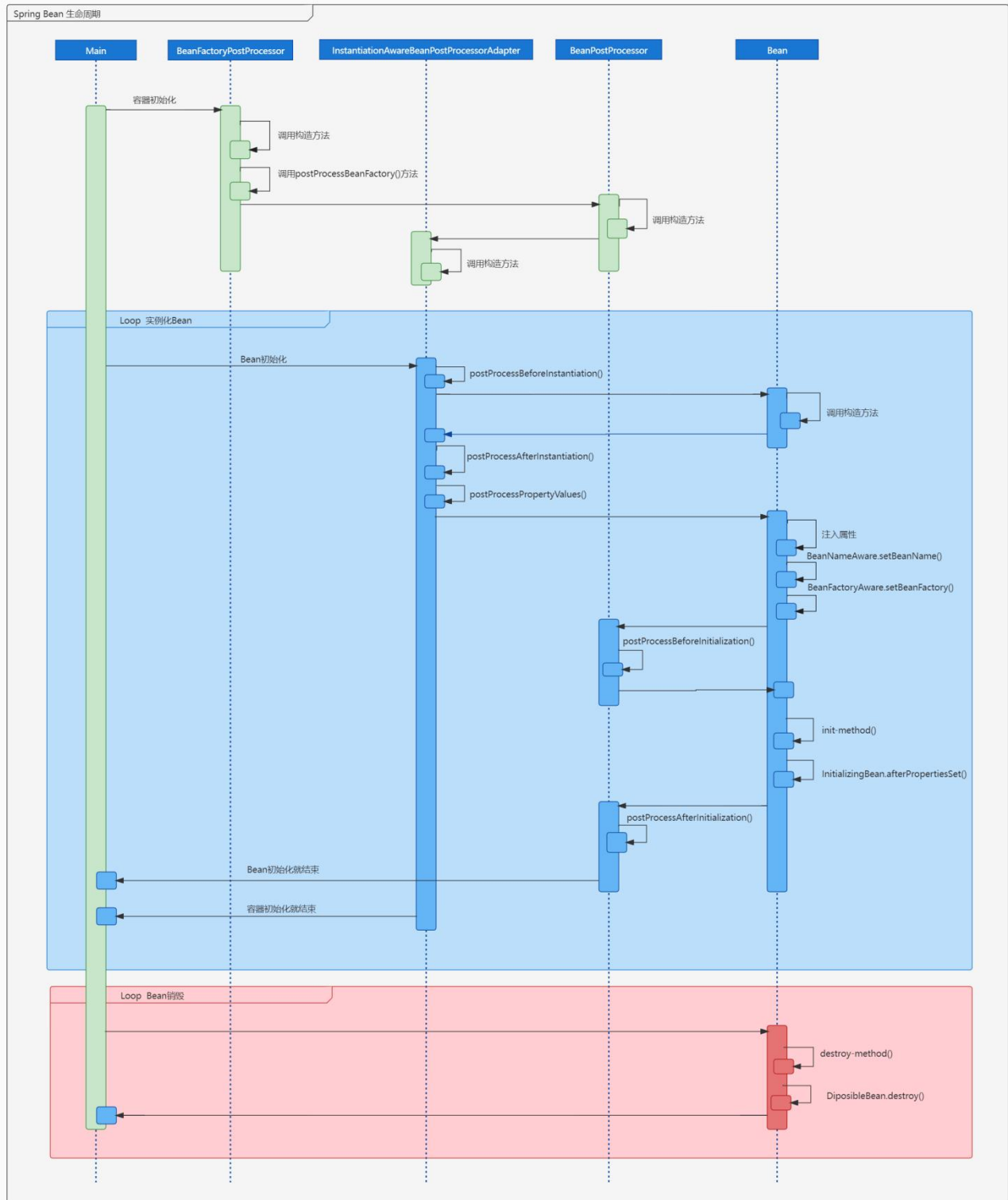
5、销毁实例阶段



这个阶段，是完成 Spring 应用上下文关闭时，将销毁 Spring 上下文中所有的 Bean。

如果 Bean 实现了 DisposableBean 接口，或者配置了 `destroy-method` 属性，将会在这个阶段被调用。

看到这里，相信大家对 Spring Bean 的生命周期有了深刻的印象，需要视频中 Spring Bean 生命周期的高清流程图，可以私信我。在附赠一张高清的时序图给大家！



我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

6. Spring 为何需要三级缓存解决循环依赖，而不是二级缓存？

今天给大家分享一道大厂面试真题，Spring 为何需要三级缓存解决循环依赖，而不是二级缓存？我一共分为五个部分来给大家介绍：

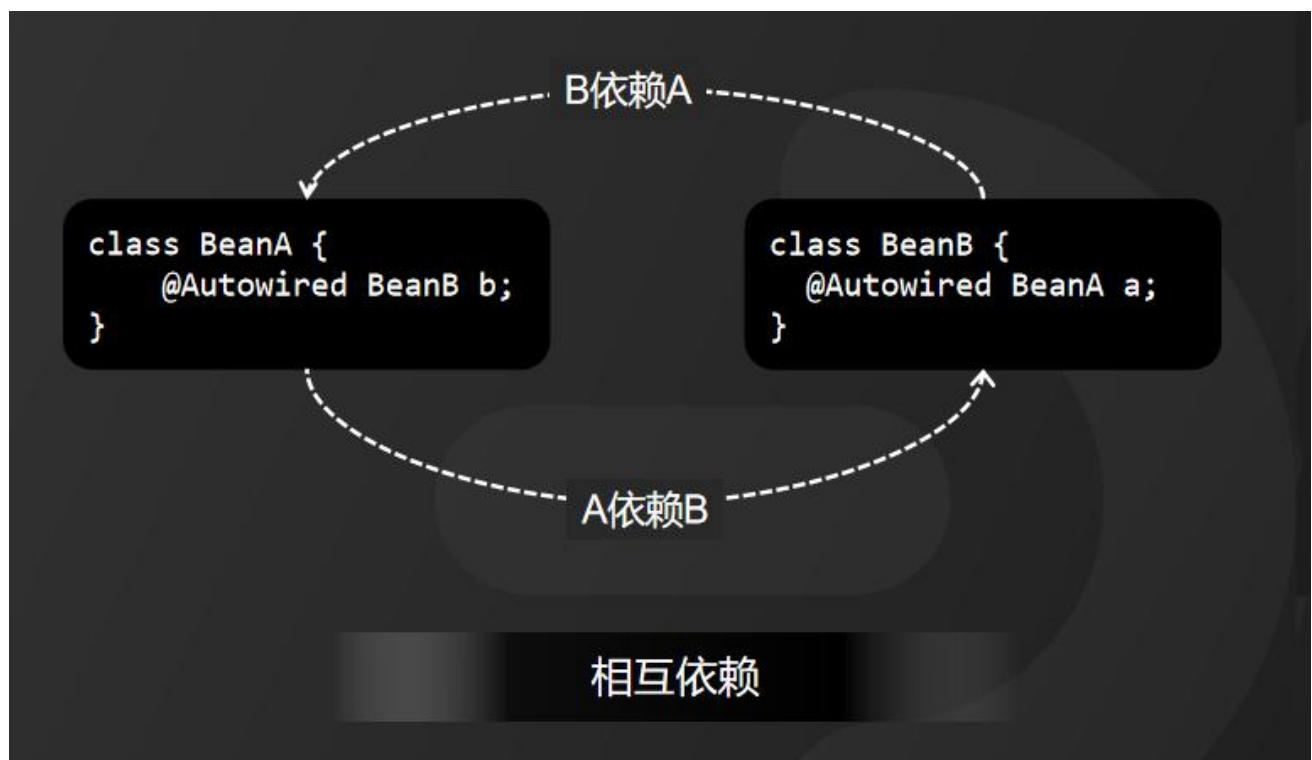
1、什么是循环依赖？

循环依赖就是指循环引用，是两个或多个 Bean 相互之间的持有对方的引用。在代码中，如果将两个或多个 Bean 互相之间持有对方的引用，因为 Spring 中加入了依赖注入机制，也就是自动给属性赋值。Spring 给属性赋值时，将会导致死循环。那么，哪些情况会出现循环依赖呢？

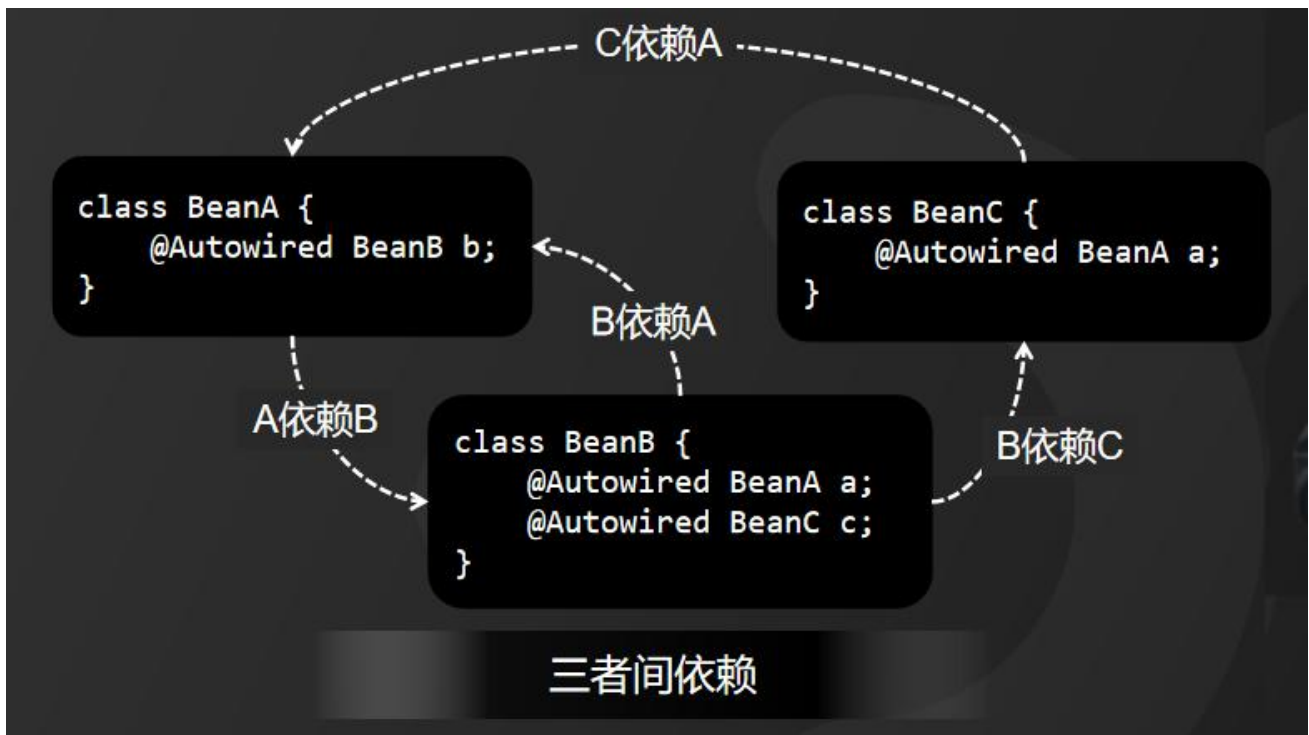
2、哪些情况会出现循环依赖？

循环依赖有三种形态：

1、相互依赖，也就是 A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



2、三者间依赖，也就是 A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



3、自我依赖，也是 A 依赖 A 形成了循环依赖自己依赖自己。



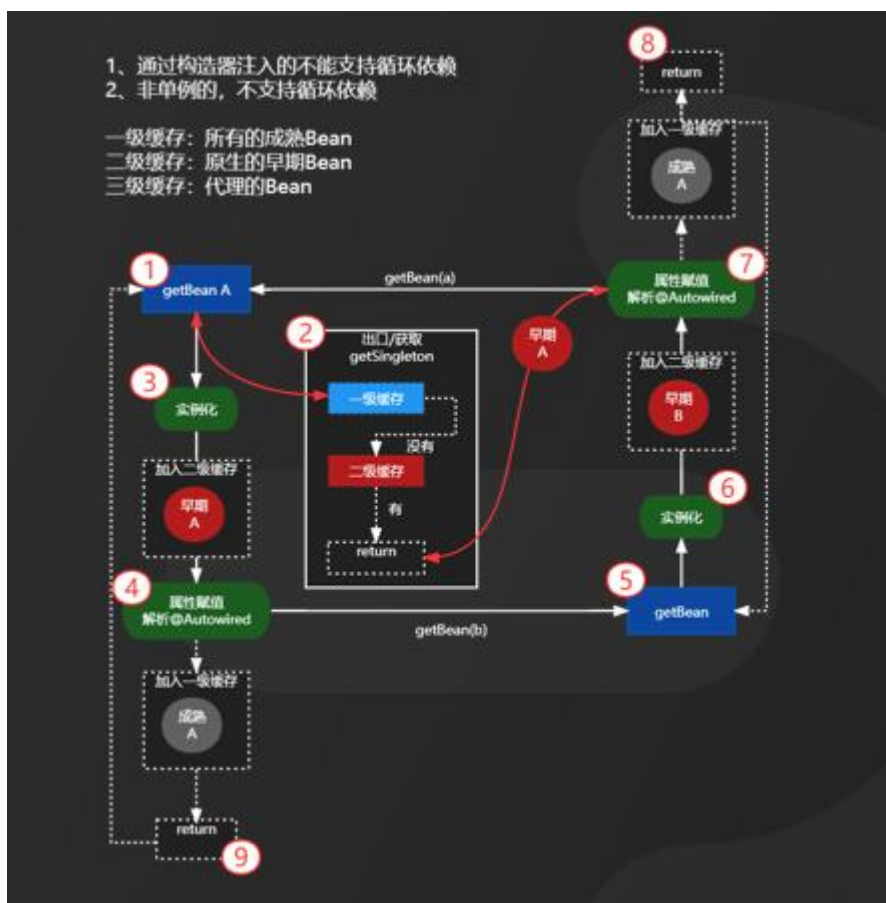
3、Spring 如何解决循环依赖问题？

Spring 中设计了三级缓存来解决循环依赖问题，当我们去调用 `getBean()` 方法的时候，Spring 会先从一级缓存中找到目标 Bean，如果发现一级缓存中没有 便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标 Bean 还没有实例化。于是，Spring 容器会实例化目标 Bean（PS：刚初始化的 Bean 称为早期 Bean），然后，将目标 Bean 放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标 Bean 存入到二级缓存，否则，便会标记该 Bean 存在循环依赖，然后将等待下一次轮询赋值，也就是解析 `@Autowired` 注解。等 `@Autowired` 注解赋值完成后（PS：完成赋值的 Bean 称为成熟 Bean），会将目标 Bean 存入到一级缓存。

总结一下，Spring 一级缓存中存放所有的成熟 Bean，二级缓存中存放所有的早期 Bean，先取一级缓存，再去二级缓存。

4、为何需要三级缓存，而不是二级缓存？

那么，前面有提到三级缓存，三级缓存的作用是什么呢？来看这样一张图，三级缓存是用来存储代理 Bean，当调用 `getBean()` 方法时，发现目标 Bean 需要通过代理工厂来创建，此时会将创建好的实例保存到三级缓存，最终也会将赋值好的 Bean 同步到一级缓存中。大家可以私信我或者在评论区留言获取高清图。



5、Spring 中哪些情况下，不能解决循环依赖问题？

1. 多例 Bean 通过 setter 注入的情况，不能解决循环依赖问题
2. 构造器注入的 Bean 的情况，不能解决循环依赖问题
3. 单例的代理 Bean 通过 Setter 注入的情况，不能解决循环依赖问题
4. 设置了@DependsOn 的 Bean 的情况，不能解决循环依赖问题

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

7. 请简述 Spring MVC 的执行流程

今天我给大家介绍一下 Spring MVC 的详细执行流程。我把 Spring MVC 的执行流程划分为三个阶段，配置阶段、初始化阶段和运行阶段。



我整理了一张完整的执行流程图，需要高清图的小伙伴可以私信我。下面详细介绍每个阶段的执行细节。

1、第一阶段：配置阶段

配置阶段，主要是完成对 xml 配置和注解配置。

具体步骤如下：

首先，从 web.xml 开始，配置 DispatcherServlet 的 url 匹配规则和 Spring 主配置文件的加载路径

然后，就是配置注解，比如@Controller、@Service、@Autowired 以及@RequestMapping 等。

2、第二阶段：初始化阶段

初始化阶段，主要是加载并解析配置信息以及 IoC 容器、DI 操作和 HandlerMapping 的初始化。

具体步骤如下：

首先，Web 容器启动以后，会由 Web 容器自动调用 DispatcherServlet 的 init() 方法。

然后，在 init() 方法中，会初始化 IoC 容器，IoC 容器其实就是个 Map。

紧接着，根据配置好的扫描包路径，扫描出相关的类，并利用反射进行实例化，存放到 IoC 容器中。

缓存之后，Spring 将再次迭代扫描 IoC 容器中的实例，给需要自动赋值的属性自动赋值。哪些属性需要自动赋值呢？比如加了@Autowired 的属性。

最后，读取@RequestMapping 注解，获得请求 url，将 url 和 Method 建立一对一的映射关系并缓存起来。我们可以简单粗暴地理解为缓存在一个 Map 中，它的 Key 就是 url，它的值是 Method。

3、第三阶段：运行阶段

运行阶段，在 Spring 启动以后，等待用户请求，完成内部调度并返回响应结果。

具体步骤如下：

用户在浏览器输入 url 之后，Web 容器会接收到用户请求。Web 容器会自动调用 doGet() 或者 doPost() 方法。从 doGet() 或者 doPost() 方法中，我们可以获得两个对象，分别是 request 和

response。通过 request 可以获得用户请求带过来的信息，通过 response 可以往浏览器端输出响应结果。

然后，根据 request 中获得的请求 url，可以从 HandlerMapping 中找到对应 Method。

接着，还是利用反射调用方法，将获得方法调用的返回结果。

最后，将返回结果通过 response 输出到浏览器，用户就可以看到响应结果。

都已经看到这里了，大家是不是觉得 Spring MVC 执行流程非常简单？

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

8. 刚折腾完 Log4J，又爆 Spring RCE 核弹级漏洞

继 Log4J 爆出安全漏洞之后，又在深夜，Spring 的 github 上又更新了一条可能造成 RCE（远程命令执行漏洞）的问题代码，随即在国内的安全圈炸开了锅。有安全专家建议升级到 JDK 9 以上，有些专家又建议回滚到 JDK 7 以下，一时间小伙伴们不知道该怎么办了。大家来看一段动画演示，怎么改都是“将军”。



大家不要慌，我给大家先临时支个招，后面再出教程。首先教大家怎么排查哪些项目存在风险，然后，再介绍修复方案。

1、第一步：排查方法

排查的主要目的是确定你的项目是否使用了 Spring 框架。当然，你的项目有没有使用 Spring 框架开发者都知道。但是，如果公司项目比较多，为了规避风险，还要对一些老项目要进行排查。那老项目如何确定是否使用了 Spring 框架呢？

方法很简单，不管你的项目是用 war 独立部署还是用 jar 包独立部署，只需要对应的 war 或者 jar 包，将后缀改为 zip 包，然后将 zip 解压。在解压后的目录中搜索是否存在 spring-beans-开头的 jar 包或者 CachedInrospectionResults.class 文件。如果存在就可以确定该项目使用了 Spring。

确定项目使用了 Spring 框架以后，如何来修复可能存在的风险呢？

2、第二步：修复方案

目前为止，Spring 官方还没有给出解决方案。我先教大家一个简单粗暴的方案，可以临时解决问题。

1、如果安装了 WAF 防护，也就是 Web 应用防火墙，只需要追加这样一个防护规则“class.*Class.***.class.***.Class”，防止远程下载。

2、如果没有安装 WAF，只需要在拦截器中增加对 class.*Class.***.class.***.Class 后缀请求的拦截就可以了。

按照这两步操作完之后，大家记得对业务运行情况进行测试，避免对已有功能造成影响。此次爆出的漏洞，攻击原理和之前 Log4J 爆出的漏洞原理差不多。如果大家对原理有兴趣的话，可以关注我发布的其他视频。

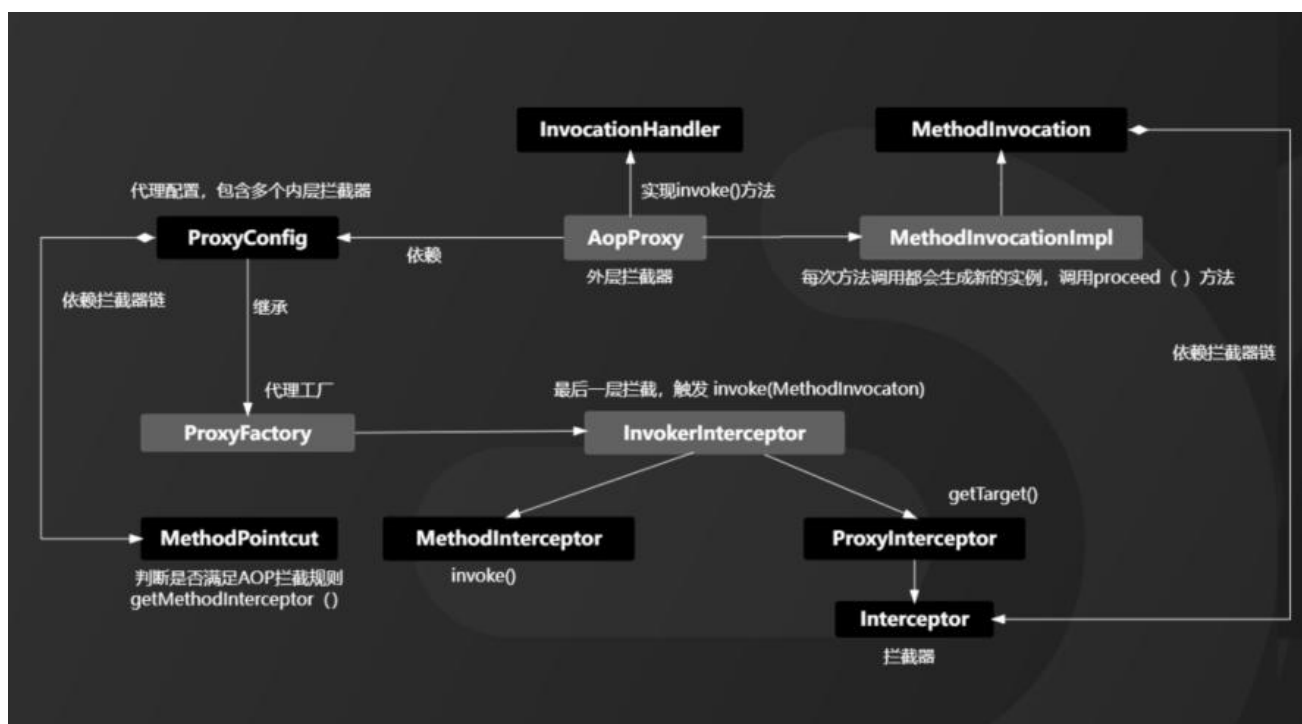
我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，技术不再难！

9. 被面试官问烂的 Spring AOP 原理，你是怎么答的？

Spring AOP 在 Spring 体系中深不可测，Spring AOP 原理也是经常在互联网大厂面试时被问到，今天，我给大家抽丝剥茧，详细到你无法想象。我划分为四个阶段给大家介绍：创建代理对象阶段、拦截目标对象阶段、调用代理对象阶段、调用目标对象阶段。



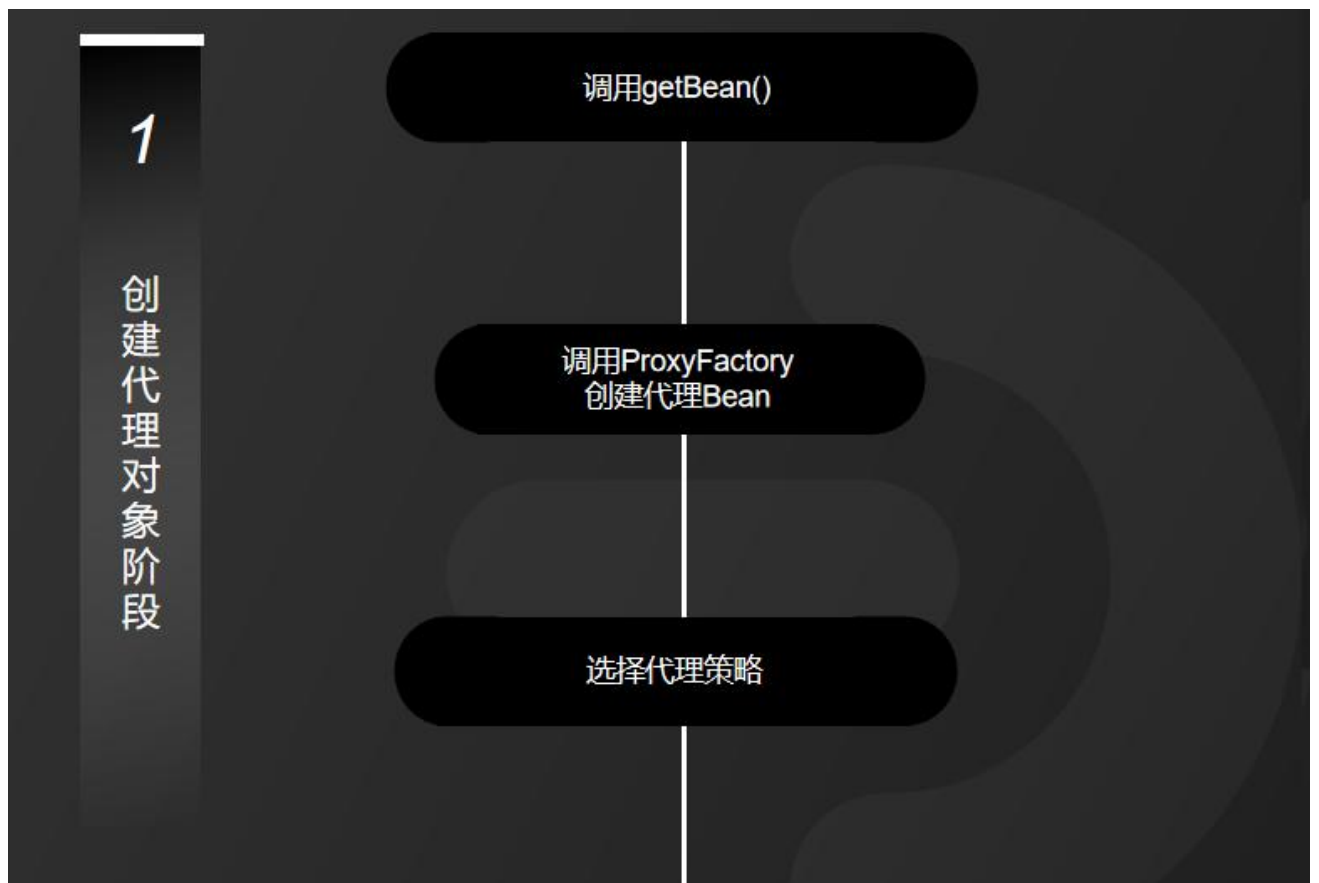
我整理了一张完整的 Spring AOP 设计原理 UML 图，需要高清图的小伙伴可以私信我。



下面详细介绍每个阶段的执行细节。

1、第一阶段：创建代理对象阶段

在 Spring 中，创建 Bean 实例都是从 `getBean()` 方法开始的，在实例创建之后，Spring 容器将根据 AOP 的配置去匹配目标类的类名，看目标类的类名是否满足切面规则。如果满足满足切面规则，就会调用 **ProxyFactory** 创建代理 Bean 并缓存到 IoC 容器中。根据目标对象的自动选择不同的代理策略。如果目标类实现了接口，Spring 会默认选择 JDK Proxy，如果目标类没有实现接口，Spring 会默认选择 Cglib Proxy，当然，我们也可以通过配置强制使用 Cglib Proxy。



2、第二阶段：拦截目标对象阶段

当用户调用目标对象的某个方法时，将会被一个叫做 AopProxy 的对象拦截, Spring 将所有的调用策略封装到了这个对象中，它默认实现了 InvocationHandler 接口，也就是调用代理对象的外层拦截器。在这个接口的 invoke() 方法中，会触发 MethodInvocation 的 proceed() 方法。在这个方法中会按顺序执行符合所有 AOP 拦截规则的拦截器链。

2

拦截目标对象阶段

AopProxy对象拦截

调用InvocationHandler的
invoke()方法

触发MethodInvocation的
proceed()方法

3、第三阶段：调用代理对象阶段

Spring AOP 拦截器链中的每个元素被命名为 MethodInterceptor，其实就是切面配置中的 Advice 通知。这个回调通知可以简单地理解为是新生成的代理 Bean 中的方法。也就是我们常说的被织入的代码片段，这些被织入的代码片段会在这个阶段执行。



4、第四阶段：调用目标对象阶段

MethodInterceptor 接口也有一个 `invoke()` 方法，在 `MethodInterceptor` 的 `invoke()` 方法中会触发对目标对象方法的调用，也就是反射调用目标对象的方法。

4

调用目标对象阶段

执行MethodInterceptor的
invoke()方法

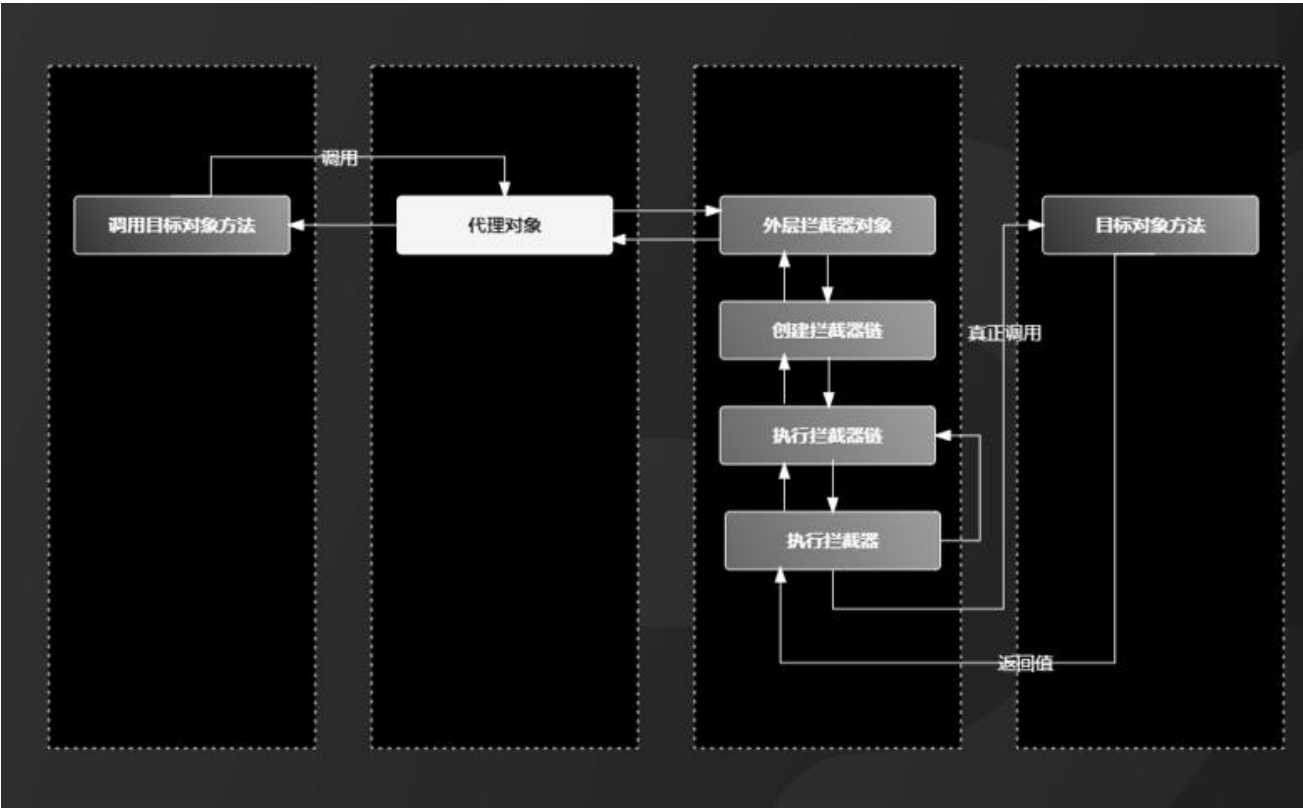
调用目标对象

Spring AOP 原理就分析到这里，最后，总结一下不迷路：

- 1、代理对象：就是由 Spring 代理策略生成的对象；
- 2、目标对象：就是我们自己写的业务代码；
- 3、织入代码：就是要在我们自己写的业务代码增加的代码片段；
- 4、切面通知：就是封装织入代码片段的回调方法；
- 5、MethodInvocation：负责执行拦截器链，在 proceed() 方法中执行；
- 6、MethodInterceptor：负责执行织入的代码片段，在 invoke() 方法中执行。

代理对象	由Spring代理策略生成的对象
目标对象	我们自己写的业务代码
织入代码	在我们自己写的业务代码增加的代码片段
切面通知	封装织入代码片段的回调方法
MethodInvocation	负责执行拦截器链，在proceed()方法中执行
MethodInterceptor	负责执行织入的代码片段，在invoke()方法中执行

都看到这里了，你还觉得 Spirng AOP 原理难吗？我再送给大家一张精简版的 Spring AOP 执行流程图，需要的小伙伴立马关注点个赞。



我是被编程耽误的文艺 Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

10.3 分钟轻松理解单线程下的 HashMap 工作原理

【引言】

HashMap 主要是用来处理键值对数据。随着 JDK 版本的更新，JDK1.8 对 HashMap 对底层也做了一些优化。今天我带大家一起来结合源码，深入浅出 HashMap 工作原理。 1

HashMap 是基于哈希表对 Map 接口的实现类，它的特点是访问数据的速度快，并不是按顺序来遍历。HashMap 提供所有可选的映射操作，但不能保证映射顺序不变，并且允许使用空值和空键。HashMap 也并不是线程安全的，当存在多个线程同时写入时，可能会导致数据不一致的情况。

1、HashMap 中的关键属性

【导航条：关键属性】

要透彻理解 HashMap 原理，首先需要对以下几个关键属性有一个基本的认识。

```
public class HashMap<K,V> extends AbstractMap<K,V>
implements Map<K,V> {

    final float loadFactor;

    int threshold;

    transient int size;

    transient int modCount;

    static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;
}
```

我们看到，HashMap 的源码片段：

第一个属性 loadFactor，它是负载因子，默认值是 0.75，表示扩容前。

第二个属性 threshold 它是记录 HashMap 所能容纳的键值对的临界值，它的计算规则是负载因子乘以 数组长度。

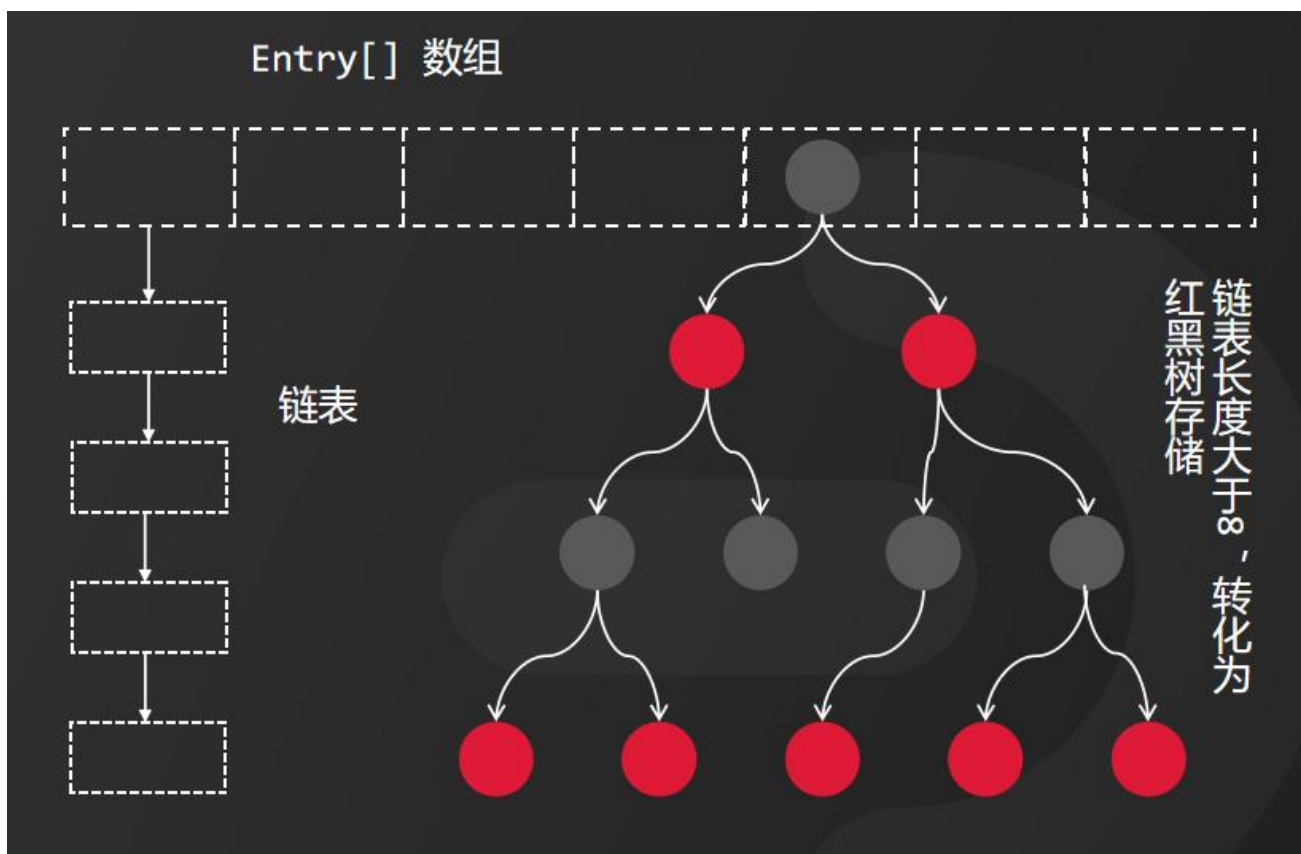
第三个属性 size，它用来记录 HashMap 实际存在的键值对的数量。

第四个属性 modCount，它用来记录 HashMap 内部结构发生变化的次数。

第五个是常量属性 DEFAULT_INITIAL_CAPACITY，它规定的默认容量是 16。

2、HashMap 的存储结构

【导航条：存储结构】



HashMap 采用的是 的存储结构。HashMap 的数组部分称为 Hash 桶，数组元素保存在一个叫做 table 的属性中。当链表长度大于等于 8 时，链表数据将会以红黑树的形式进行存储，当长度降到 6 时，又会转成链表形式存储。

```
static class Node<K,V> implements Map.Entry<K,V> {  
    ...  
    final int hash;  
    final K key;  
    V value;  
    Node<K,V> next;  
    ...  
}
```

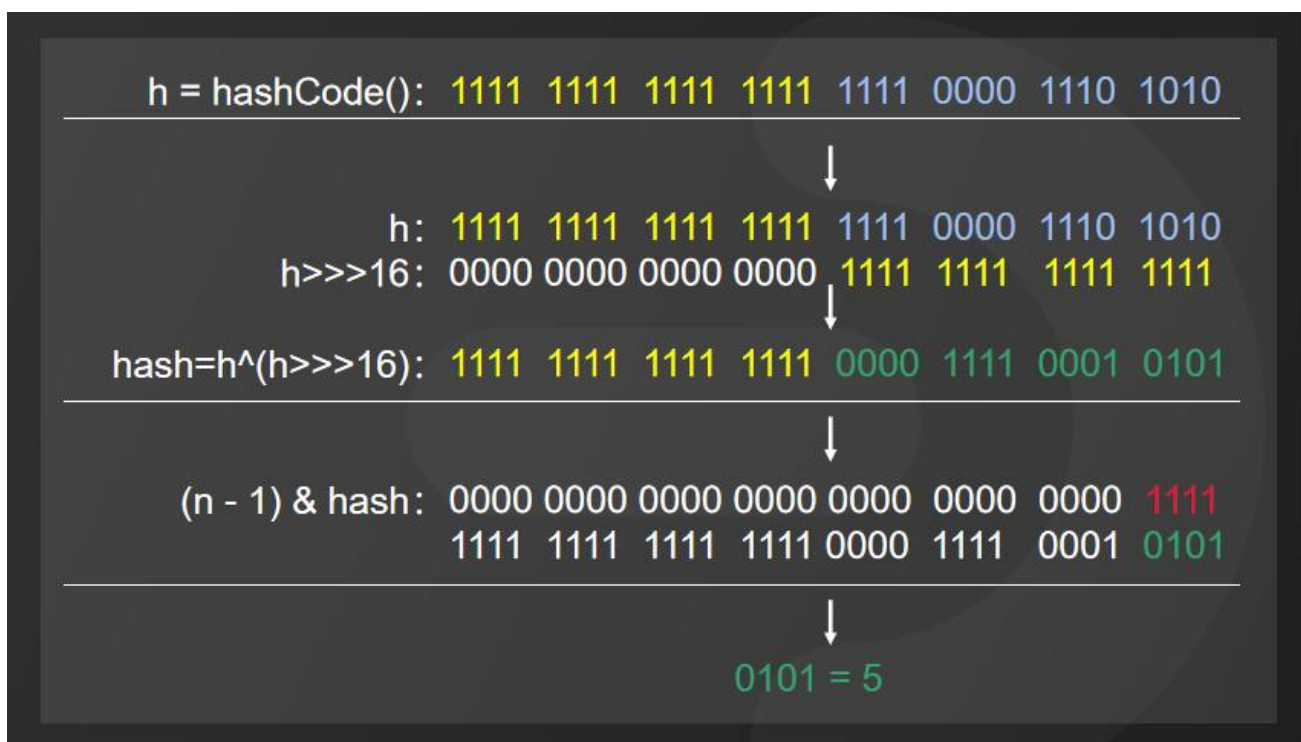
每个 Node 节点，保存了用来定位数组索引位置的 hash 值、Key、Value 和链表指向的下一个 Node 节点。而 Node 类是 HashMap 的内部类，它实现了 Map.Entry 接口，它的本质其实可以简单的理解成就是一个键值对。来看一下源码。

3、HashMap 的工作原理

【导航条：工作原理】

当我们向 HashMap 中插入数据时，首先要确定 Node 在数组中的位置。那如何确定 Node 的存储位置呢？以添加 Key 为字符串“e”的对象为例：

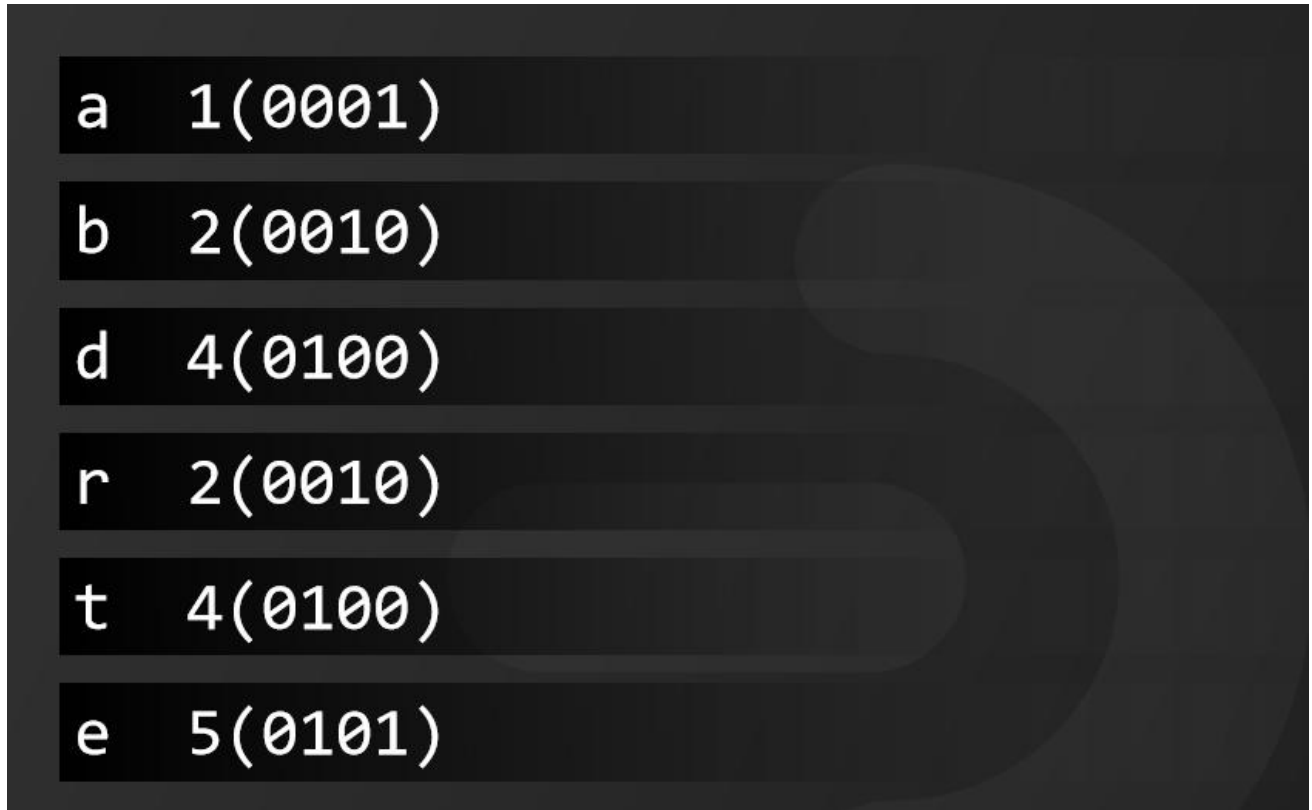
```
public static int hashCode(Object o) {  
    return o != null ? o.hashCode() : 0;  
}
```



HashMap 首先调用 hashCode() 方法，获取 Key 的 hashCode 值为 h。然后对 h 值进行高位运算；将 h 右移 16 位取得 h 的高 16 位，与 进行异或运算，最后得到 h 的值与 (table.length - 1) 进行与运算获得该对象的保留位，最后计算出下标。当然，这是最官方的描述。有的小伙伴可能已经迷糊了。其实，这段运算过程，简单地理解成求模取余法。

就是用 hash 值和数组的长度减 1，取模，最后得到数组的下标，这样可以保证数组下标不越界。只不过，位运算是二进制运算，效率更高。

最后，来看一段动画演示，假设有“a”、“b”、“d”、“r”，“t”，“e”的 Key。通过计算得到的下标分别为 1、2、4、2、4、5



它们的插入顺序如动画所示。

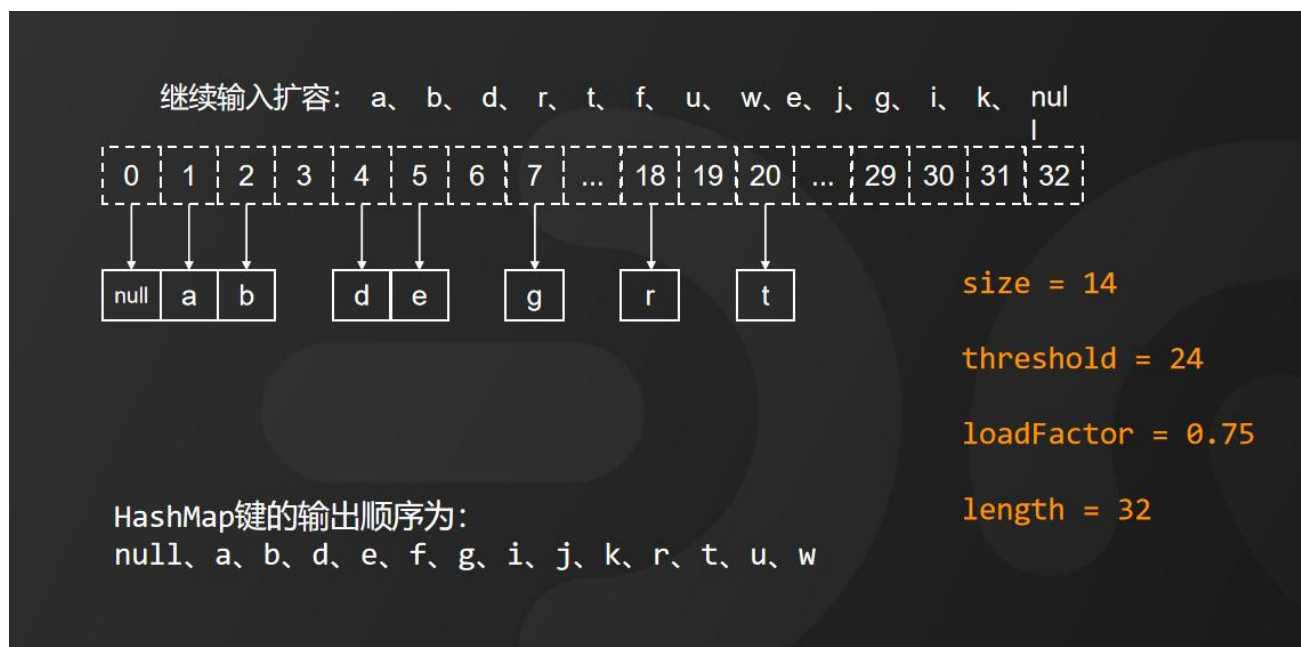


如果我们再次插入 "a", "g", "i", null 四个 Key, 来看 HashMap 的内部变化。



当插入第二个以 a 为 Key 的对象时, 会将新值赋值给 a 的值。当插入的对象大小超过临界值时, HashMap 将新建一个桶数组并重新赋值 (当然, JDK1.7 和 1.8 重新赋值的方式略有不同)

这个时候，HashMap 键的输出顺序为 null、a、b、r、d、t、e、g、i



HashMap 的工作原理，你搞懂了吗？

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11. 什么是内存溢出，什么是内存泄漏？

什么是内存溢出，什么是内存泄漏？这是很多小伙伴经常问我的一个问题，今天花 3 分钟时间给大家介绍一下。先来介绍什么是内存溢出？

1、什么是内存溢出？

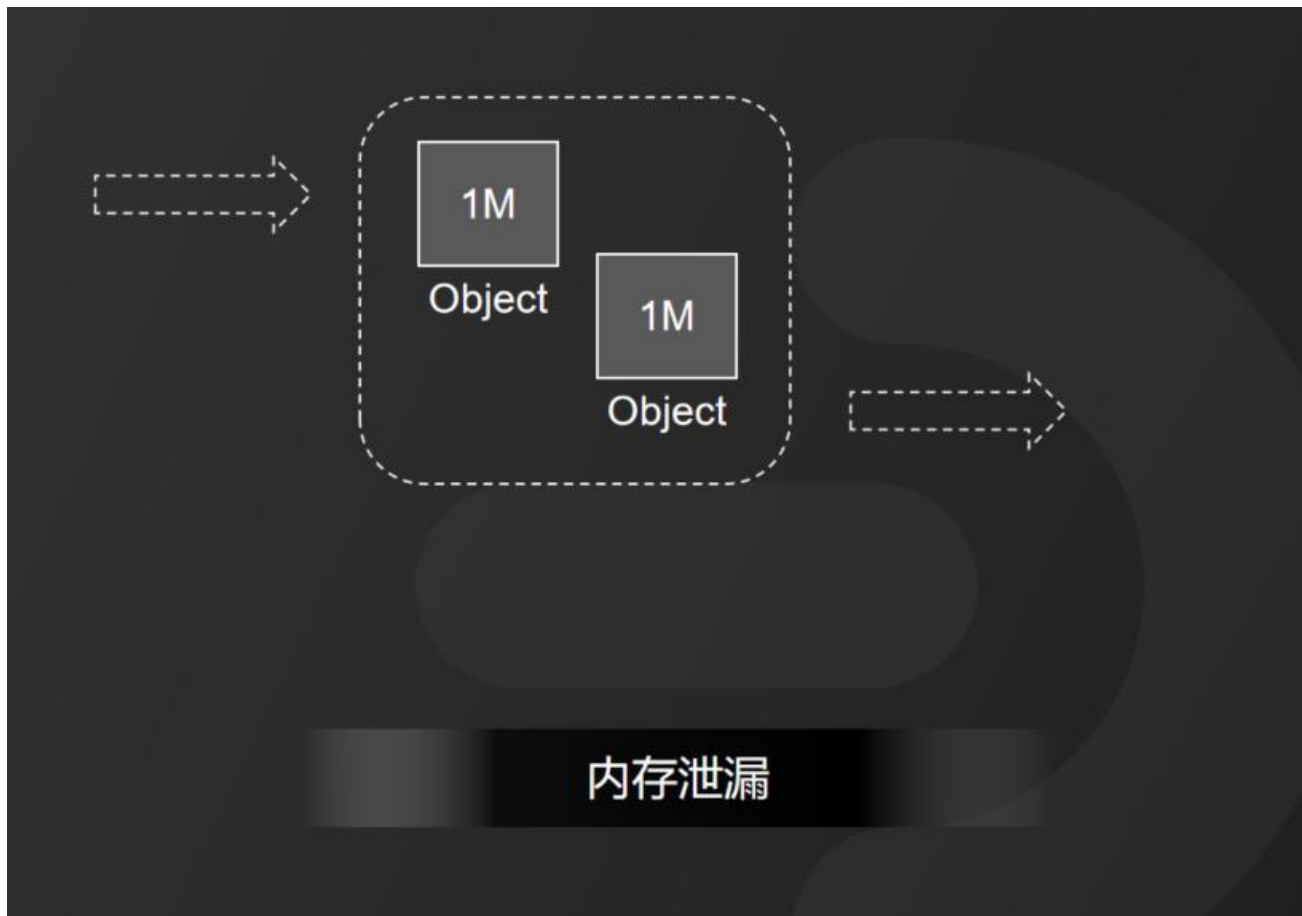
发生内存溢出。

我们来看右侧的区域，假设我们 JVM 中可用的内存空间只剩下 3M，但是我们要创建一个 5M 的对象，那么，新创建的对象就放不进去了。这个时候，我们就叫做内存溢出。就好比是一个容量只有 300ml 的水杯，我们硬要往里面倒 500ml 的水，这时候，水就会溢出，倒不进去了，这就相当于是内存的溢出。

那么，内存泄漏又是怎么回事呢？

1 2、什么是内存泄漏？

还是来看这张图，

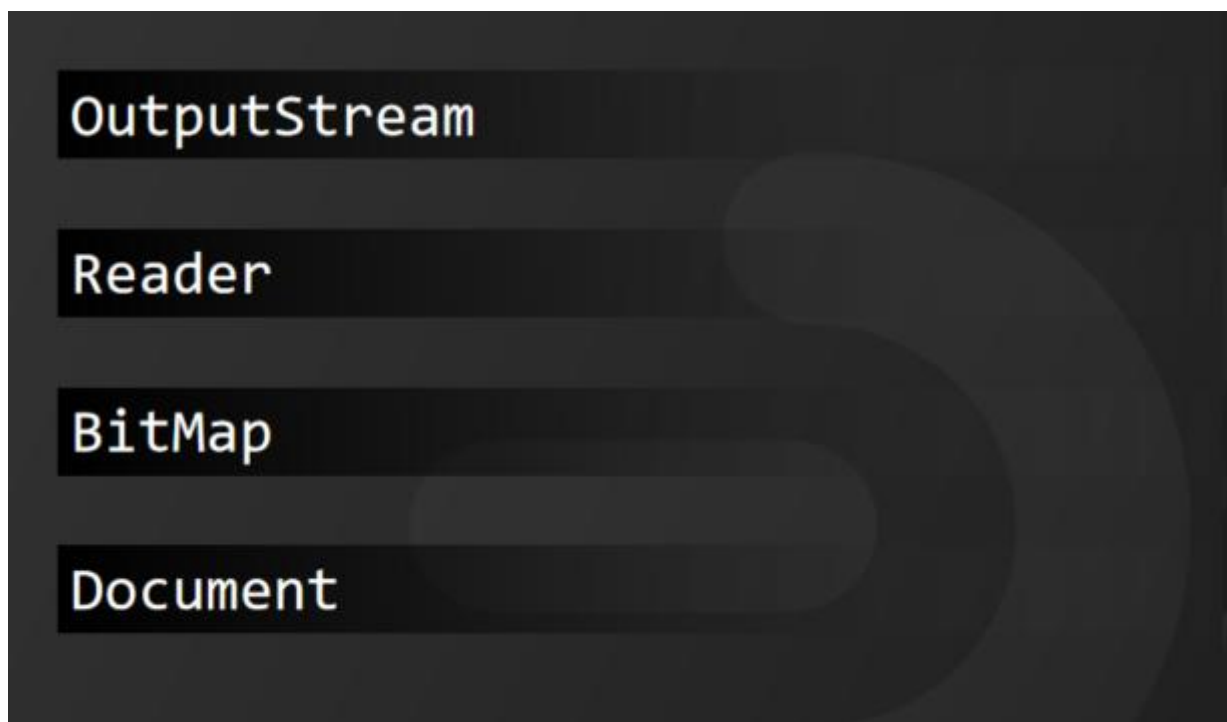


它代表业务代码执行时，所需要占用的内存空间。这段业务代码中创建了两个 1M 的对象，一起会占用 2M 内存。当对象使用完之后，这两个对象并没有释放，因此内存中会留下 2M 的内存空间一直被占用。而我们的业务代码在程序中会被反复执行，每次执行都会留下 2M 不被释放，反复执行多次之后，随着时间的累积，就会有大量的对象用完不被释放，导致这些对象不能得到回收而发生内存溢出，这种情况就叫做内存泄漏。

也就是说，在我们的业务代码执行过程中，有些对象它应该被回收，但是又有其他对象引用引用它，因此，GC 不能自动回收。所以，该回收的垃圾对象没有被回收，垃圾对象越堆越多，可用内存越来越少，若可用内存无法存放新的垃圾对象，最终导致内存泄漏。内存泄漏最终会导致内存溢出。

3、如何避免？

我们在 Code 过程中，特别是些一些流对象，



比如 OutputStream, Reader, BitMap, Document, 很容易忘了 Close。最麻烦的是还要顺序回收, 顺序错了还产生空指针, 所以, 大家在 Code 过程一定要注意, 当然, 现在有很多 IDE 会智能提示, 也避免了很多低级错误。

以上就是我对内存溢出和内存泄漏的分析, 听懂了的小伙伴, 请关注点个赞, 下次不迷路。

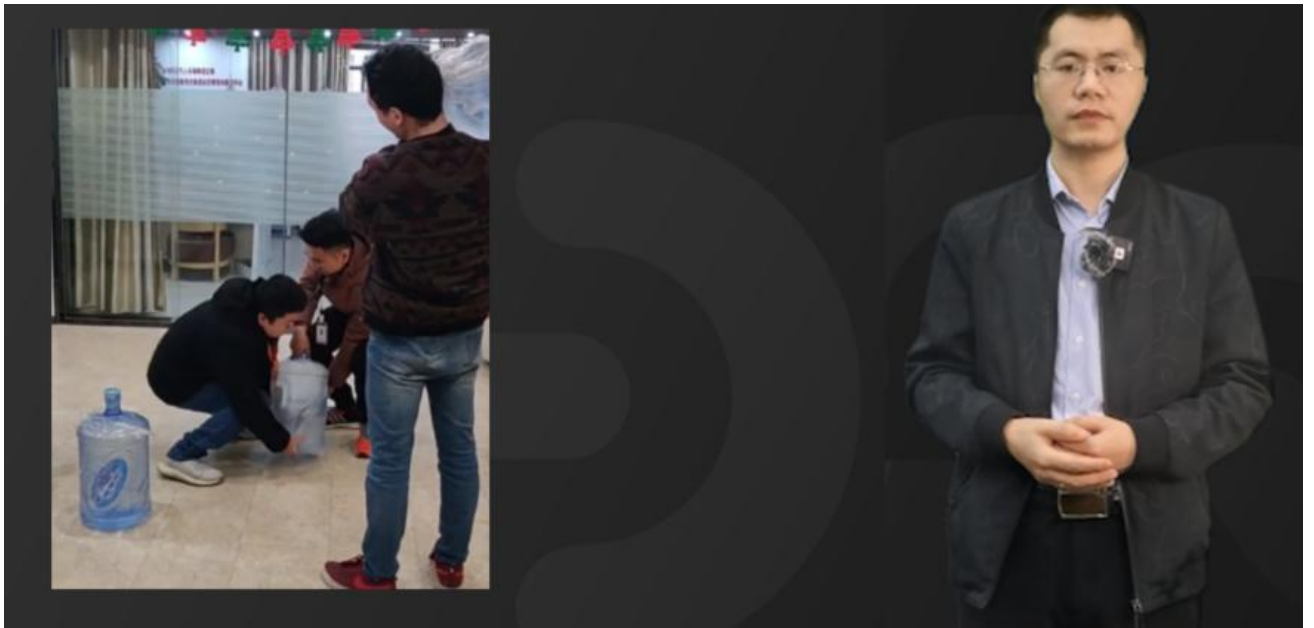
我是被编程耽误的文艺 Tom, 如果大家还有其他疑问, 请在评论区留言。如果本次面试解析对你有帮助, 请动动手指一键三连分享给更多的人。关注我, 面试不再难!

12. 什么条件下会产出死锁, 如何避免死锁?

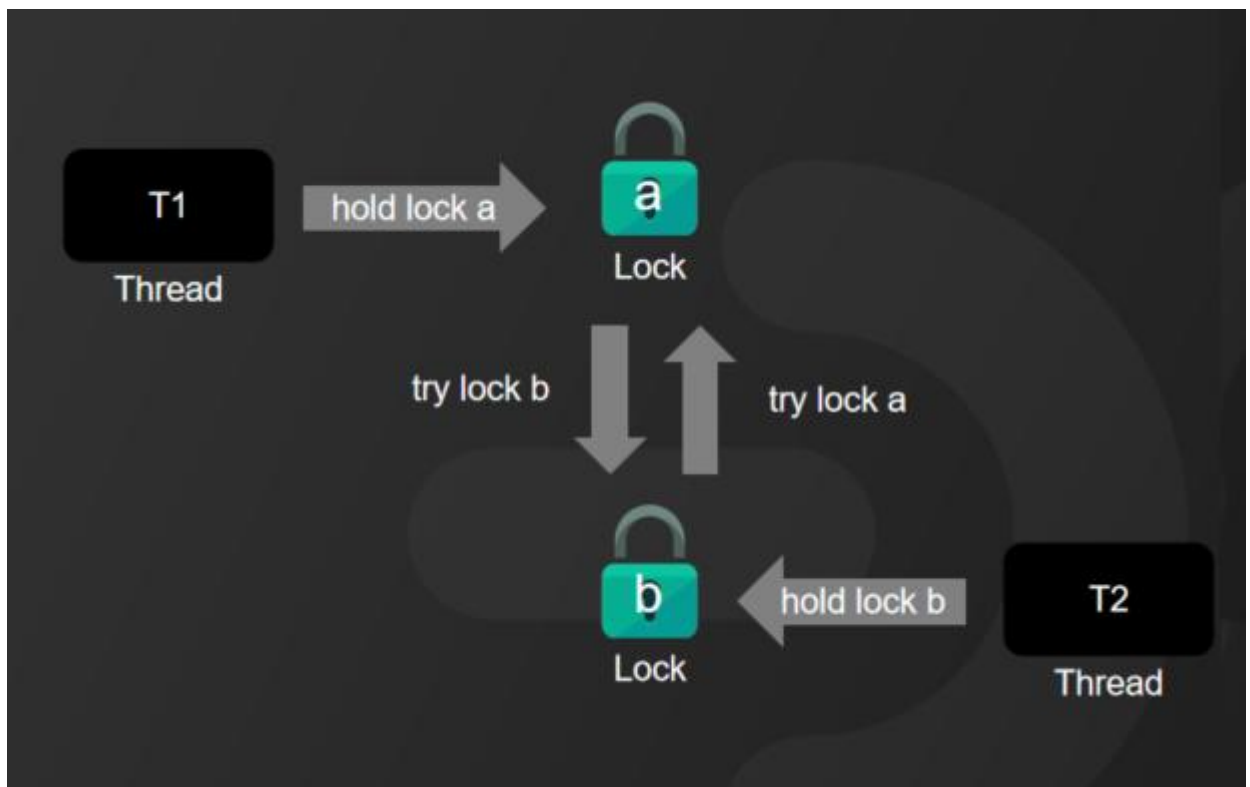
一个去美团面试的小伙伴私我说, 被面试官问到一个死锁的问题难倒了, 面试前还特意刷了题, 面试的时候就是脑子一片空白不知道怎么回答。今天, 我给大家彻底讲明白。

1、什么是死锁?

先来看这样一段视频,



看完你也许已经懂了，那到底什么是死锁呢？



死锁，简单来说就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致相互等待的现象。如果没有外部干预，线程会一直处于阻塞状态，无法往下执行。这样一直等待处于阻塞状态的线程，被称为死锁线程。

2、产生死锁的原因

产生死锁需要同时满足以下四个条件：

互斥条件	共享资源a和b只能被一个线程占用
请求和保持条件	线程T1已经获取共享资源a，在等待共享资源b的时候，不释放共享资源a
不可抢占条件	其他线程不能强行抢占线程T1占有的资源
循环等待条件	线程T1等待线程T2占有的资源，线程T2等待线程T1占有的资源，这形成了循环等待

第一个：互斥条件，共享资源 a 和 b 只能被一个线程占用；

第二个：请求和保持条件，线程 T1 已经获取共享资源 a，在等待共享资源 b 的时候，不释放共享资源 a；

第三个：不可抢占条件，其他线程不能强行抢占线程 T1 占有的资源；

第四个：循环等待条件，线程 T1 等待线程 T2 占有的资源，线程 T2 等待线程 T1 占有的资源，这形成了循环等待。

3、如何避免死锁？

线程产生死锁之后，只能通过外部干预来解决问题，比如重启程序，或者 Kill 线程。所以，我们只能在写代码时规避死锁的产生。那么如何避免死锁产生呢？根据产生死锁的四个必要条件，我们只需要破坏其中任何一个条件就可以解决。

破坏互斥条件	无法破坏
破坏互斥条件	在首次执行时一次性申请所有的资源
破坏不可抢占条件	主动释放线程占有的资源
破坏循环等待条件	按序申请资源来预防死锁的产生

第一个互斥条件是没有办法被破坏的，因为它是互斥锁的基本约束。其他三个条件都可以通过人工干预来破坏。比如请求保持条件，我们可以在首次执行一次性申请所有的资源，这样就不存在等待锁的问题了。

第二个，对于不可抢占条件来说，占用部分资源的线程在进一步申请其他资源的时候如果申请不到，我们可以主动释放它占有的资源。这样不可抢占这个条件就被破坏了。

第三个，对于循环等待条件来说，可以通过按序申请资源来预防死锁的产生。所谓按序申请，就是给资源编号，所有线程可以按照线性化的序号顺序去申请共享资源，先申请需要序号小的，再申请序号大的，这样循环等待自然就不存在了。

以上就是我对死锁的理解，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

13. 两个 Integer 对象比较大小，为什么 100 等于 100, 1000 不等于 1000 ？

前几天，有位小伙伴向我反馈，在维护代码过程中，出现了一个莫名其妙的问题。明明上线之后程序跑得还好好，可程序上线运行一段时间之后，所有，代码没有做任何修改，发 `cxcccc` 现运

行结果和期望值恰好相反。因为涉及到金额造成了比较大的损失，最后，这位小伙伴还被公司辞退了，大家可以来评论一下，这位小伙伴背的这个锅值不值？

1、业务场景

大家来看，他的代码大致是这样写的：

```
public boolean xxx(Integer a,Integer b){  
    ...  
    if(a == b){  
        ...  
        return true;  
    }  
    ...  
    return false;  
}
```

a和b输入100，返回结果是：true

a和b输入1000，返回结果是：false

一般情况下，a 和 b 都输入 100 的时候，返回为 true，但当 a 和 b 都输入 1000 的时候，返回为 false。按照正常逻辑理解，100 等于等于 100，那 1000 为什么不等于等于 1000 呢？这位同学，百思不得其解。于是，这位同学，还特意写了一段测试代码

```
Integer a = 100,b = 100,c = 1000,d = 1000;  
  
System.out.println((a == b) + “,” + (c == d));
```

运行结果是：true,false

这到底是什么原因呢？我们对照 Integer 的源码来进行分析：

2、源码分析

我摘取了 Integer 的源码片段，它有一个 valueOf() 的方法：

```
public final class Integer extends Number implements Comparable<Integer> {  
    ...  
  
    public static Integer valueOf(int i) {  
        if (i >= IntegerCache.low && i <= IntegerCache.high)  
            return IntegerCache.cache[i + (-IntegerCache.low)];  
        return new Integer(i);  
    }  
    ...  
}
```

我们可以看到，Integer 源码中的 valueOf() 方法做了一个条件判断，其中 IntegerCache.low 的值为-128，

```
public final class Integer extends Number implements Comparable<Integer> {  
    ...  
  
    public static Integer valueOf(int i) {  
        if (i >= IntegerCache.low && i <= IntegerCache.high)  
            return IntegerCache.cache[i + (-IntegerCache.low)];  
        return new Integer(i);  
    }  
    ...  
}
```

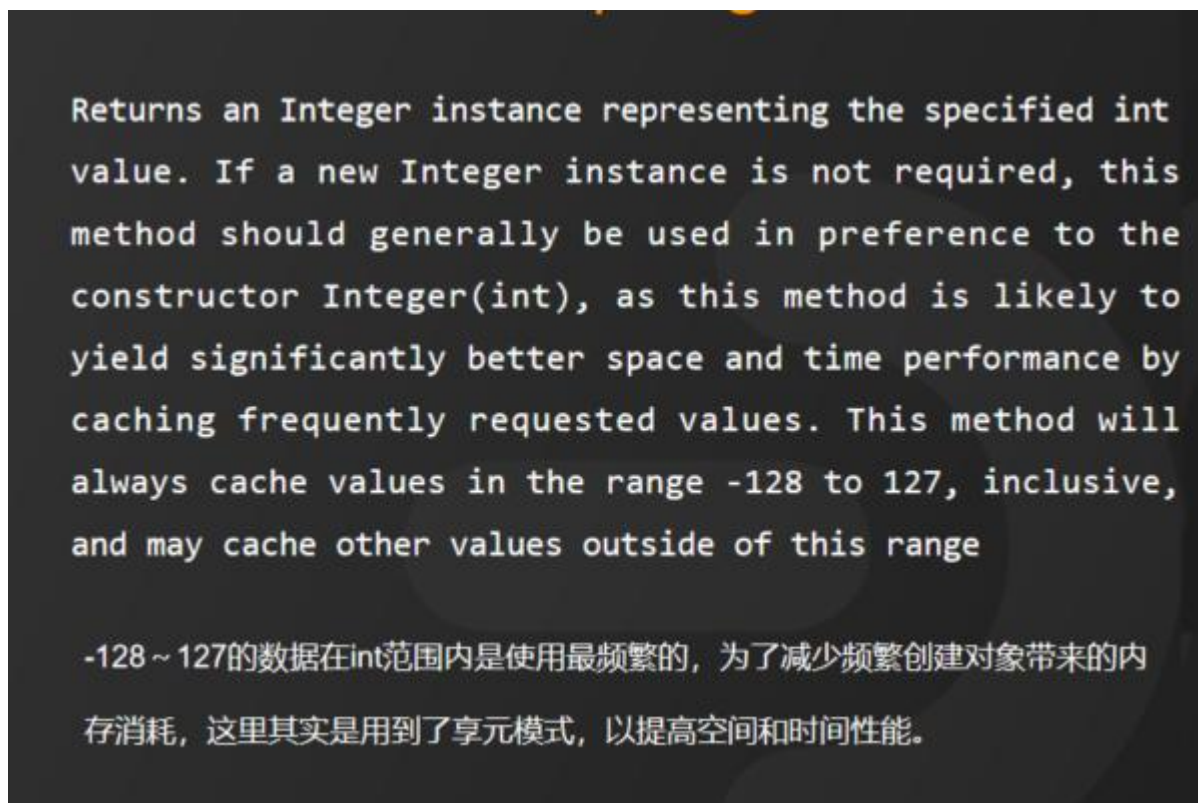
值为-128

IntegerCache.high 的值为 127。



也就是说如果目标值在-128~127 之间，会直接从 cache 数组中取值，否则就会新建对象。

这里又有人会问了，那为什么默认是-128 - 127，怎么不是-200 - 200 或者是其他值呢？那 JDK 为何要这样做呢？



在 Java API 中是这样解释的：

Returns an Integer instance representing the specified int value. If a new Integer instance is not required, this method should generally be used in preference to the constructor Integer(int), as this method is likely to yield significantly better space and time performance by caching frequently requested values. This method will always cache values in the range -128 to 127, inclusive, and may cache other values outside of this range

大致意思是：

-128~127 的数据在 int 范围内是使用最频繁的，为了减少频繁创建对象带来的内存消耗，这里其实是用到了享元模式，以提高空间和时间性能。

在 JDK 中，这样的应用不止 int，我给小伙伴们整理了一个表格，表格中的其他类型也都应用了享元模式，也就是说对数值做了缓存，只是缓存的范围不一样，具体如表中所示：

基本类型	大小	最小值	最大值	包装器类型	缓存范围	是否支持自定义
boolean	-	-	-	Boolean	-	-
char	6bit	Unicode 0	Unicode 2(16)-1	Character	0~127	否
byte	8bit	-128	127	Byte	-128~127	否
short	16bit	-2(15)	2(15)-1	Short	-128~127	否
int	32bit	-2(31)	2(31)-1	Integer	-128~127	支持
long	64bit	-2(63)	2(63)-1	Long	-128~127	否
float	32bit	IEEE754	IEEE754	Float	-	
double	64bit	IEEE754	IEEE754	Double	-	
void	-	-	-	Void	-	-

我需要这张表的小伙伴们可以私信我，以上就是关于 Integer 对象比较大小的分析，听懂了的小伙伴们，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

14. 为什么 HashMap 会产生死循环？

HashMap 死循环是一个比较常见、也是比较经典的面试题，在大厂的面试中也经常被问到。HashMap 的死循环问题只在 JDK1.7 版本中会出现，主要是 HashMap 自身的工作机制，再加上并发操作，从而导致出现死循环。JDK1.8 以后，官方彻底解决了这个问题。

1、数据插入原理

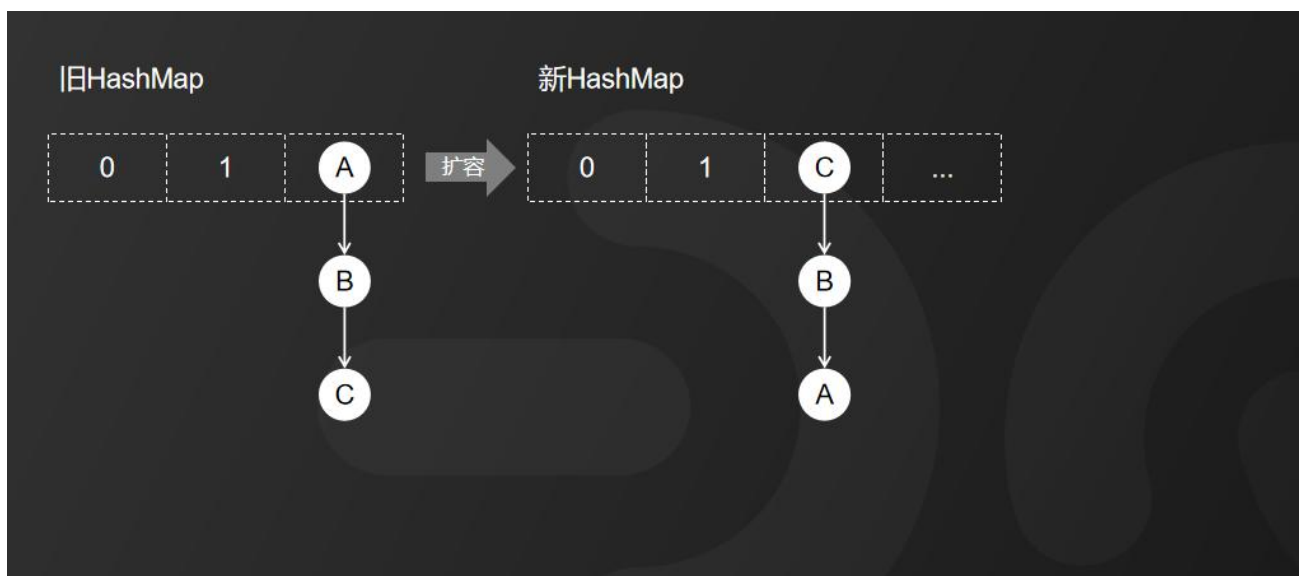
在分析原因之前，我先带大家了解一下 JDK1.7 中 HashMap 插入数据的原理，来看动画演示：



由于 JDK 1.7 中 HashMap 的底层存储结构采用的是数组 加 链表的方式。



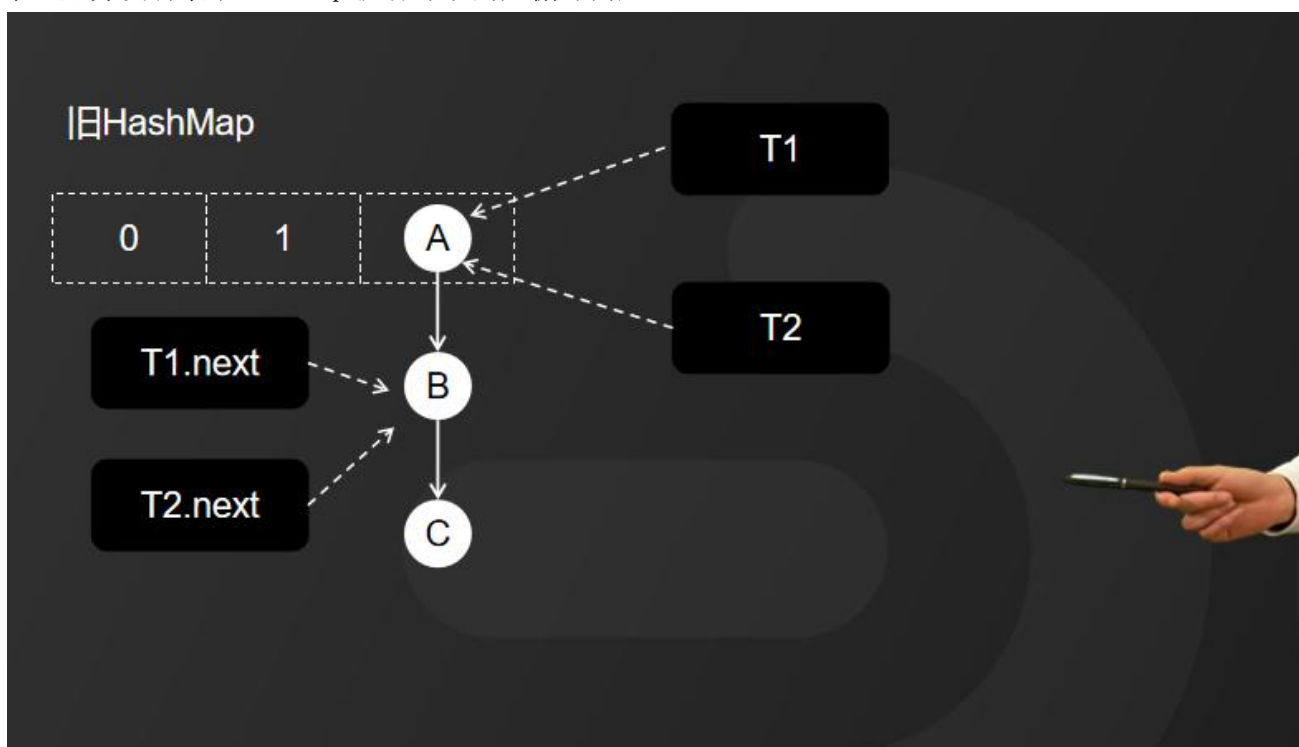
而 HashMap 在数据插入时又采用的是头插法，也就是说新插入的数据会从链表的头节点进行插入。



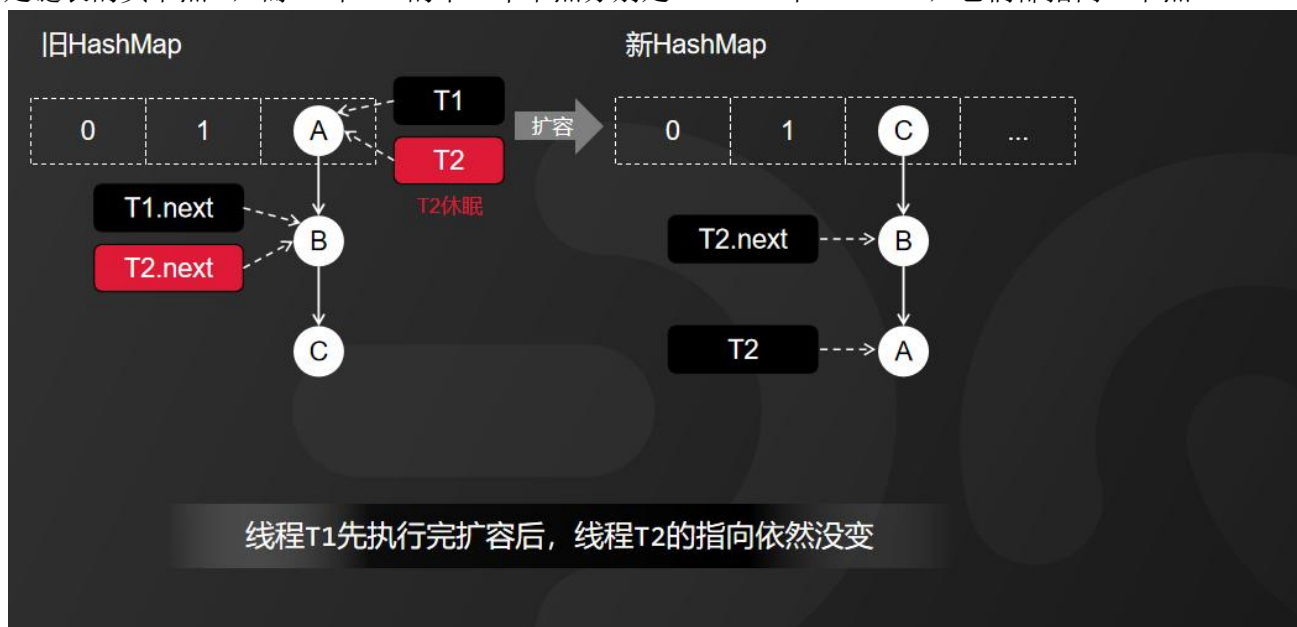
因此，HashMap 正常情况下的扩容就是这样一个过程。我们来看，旧 HashMap 的节点会依次转移到新的 HashMap 中，旧 HashMap 转移链表元素的顺序是 A、B、C，而新 HashMap 使用的是头插法插入，所以，扩容完成后最终在新 HashMap 中链表元素的顺序是 C、B、A。

2、导致死循环的原因

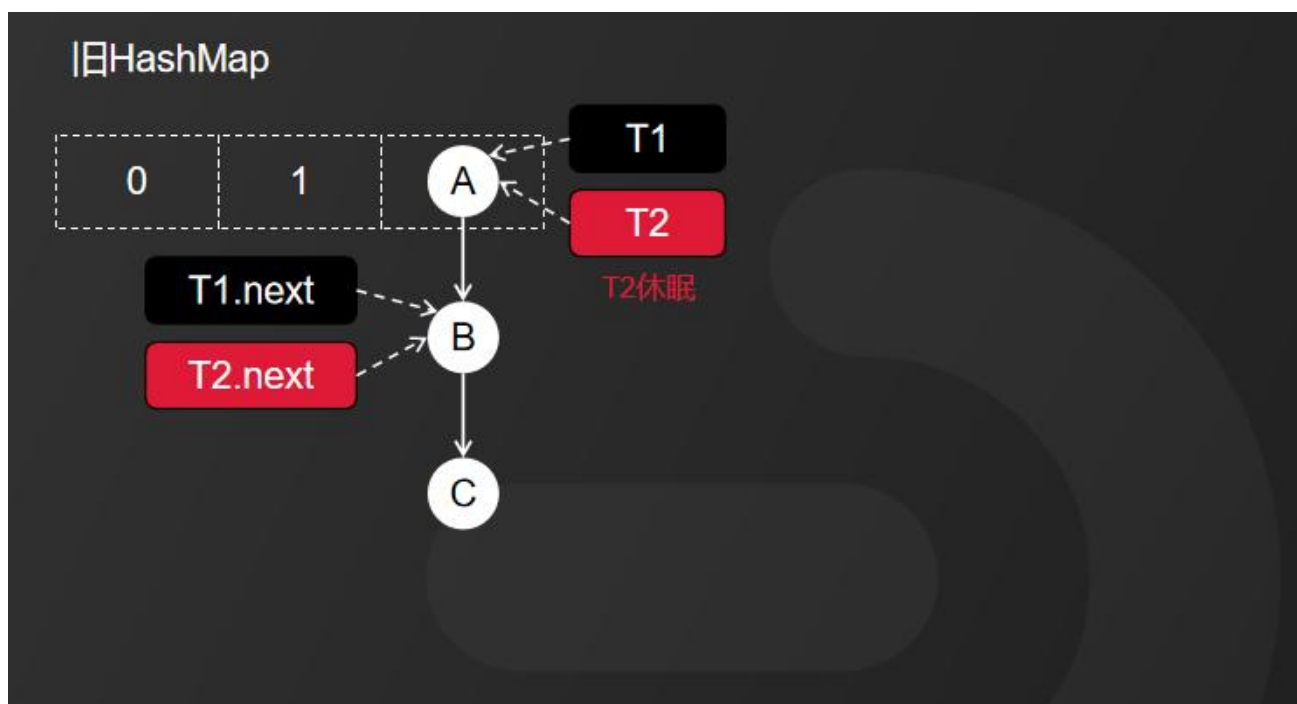
接下来，我通过动画演示的方式，带大家彻底理解造成 HashMap 死循环的原因。我们按以下三个步骤来还原并发场景下 HashMap 扩容导致的死循环问题：



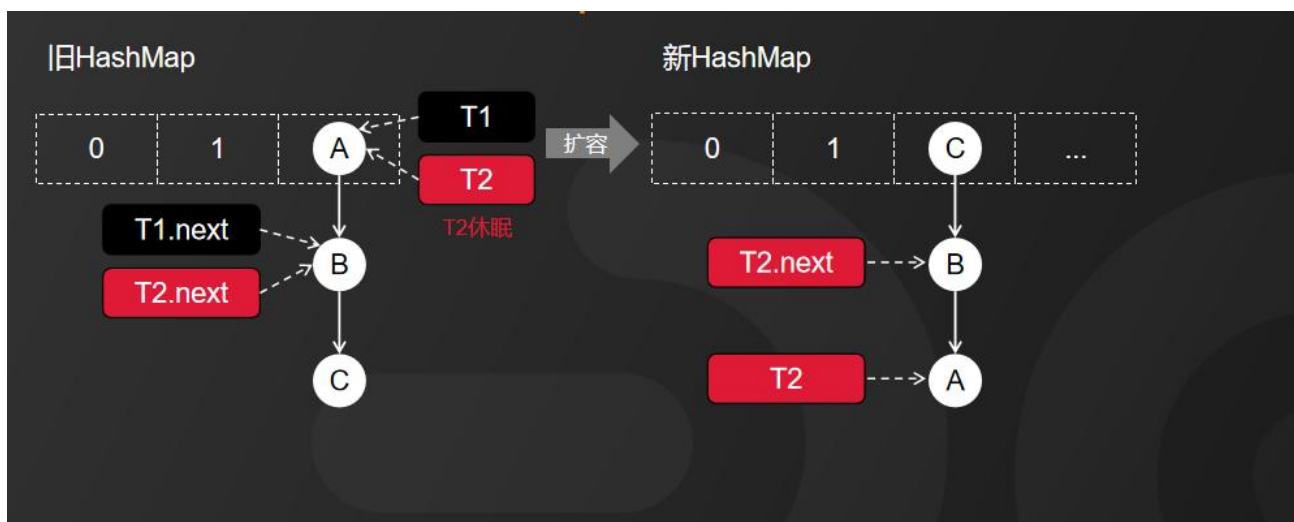
第一步：线程启动，有线程 T1 和线程 T2 都准备对 HashMap 进行扩容操作，此时 T1 和 T2 指向的都是链表的头节点 A，而 T1 和 T2 的下一个节点分别是 T1.next 和 T2.next，它们都指向 B 节点。



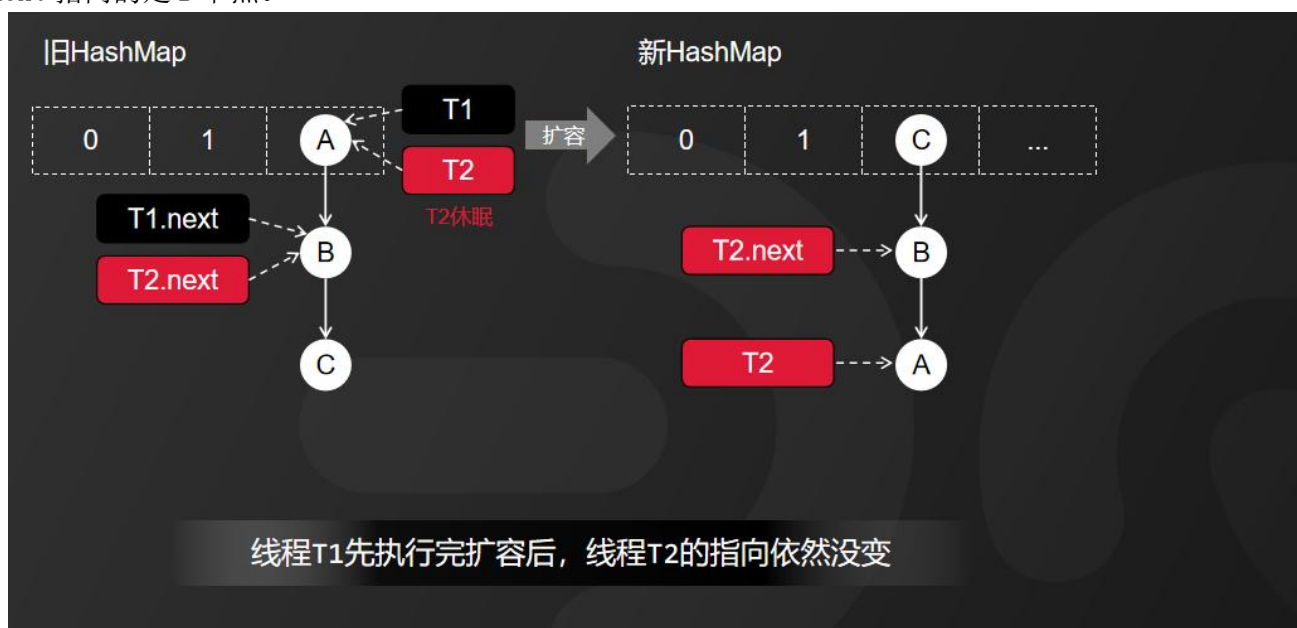
第二步：开始扩容，这时候，假设线程 T2 的时间片用完，进入了休眠状态，而线程 T1 开始执行扩容操作，一直到线程 T1 扩容完成后，线程 T2 才被唤醒。



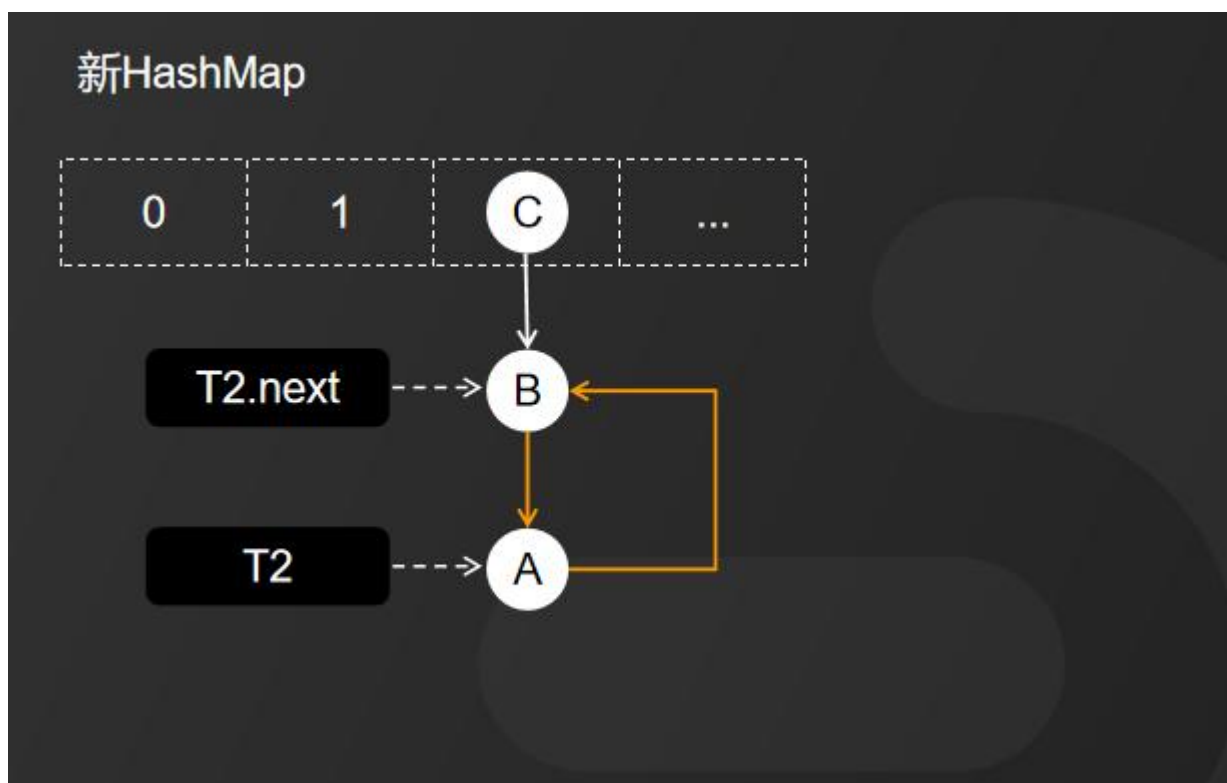
T1 完成扩容之后的场景就变成动画所示的这样。



因为 HashMap 扩容采用的是头插法，线程 T1 执行之后，链表中的节点顺序发生了改变。但线程 T2 对于发生的一切还是不可知的，所以它指向的节点引用依然没变。如图所示，T2 指向的是 A 节点，T2.next 指向的是 B 节点。



当线程 T1 执行完成之后，线程 T2 恢复执行时，死循环就发生了。



因为 T1 执行完扩容之后，B 节点的下一个节点是 A，而 T2 线程指向的首节点是 A，第二个节点是 B，这个顺序刚好和 T1 扩容之前的节点顺序是相反的。T1 执行完之后的顺序是 B 到 A，而 T2 的顺序是 A 到 B，这样 A 节点和 B 节点就形成了死循环。

3、解决方案

避免 HashMap 发生死循环的常用解决方案有三个：

- 1)、使用线程安全的 ConcurrentHashMap 替代 HashMap，个人推荐使用此方案。
- 2)、使用线程安全的容器 Hashtable 替代，但它性能较低，不建议使用。
- 3)、使用 synchronized 或 Lock 加锁之后，再进行操作，相当于多线程排队执行，也会影响性能，不建议使用。

4、总结

HashMap 死循环只发生在 JDK1.7 版本中，主要原因是 JDK1.7 中的 HashMap，在头插法 加 链表 加 多线程并发 加 扩容这几个情形累加到一起就会形成死循环。多线程环境下建议采用 ConcurrentHashMap 替代。在 JDK1.8 中，HashMap 改成了尾插法，解决了链表死循环的问题。

以上就是关于 HashMap 死循环原因的分析，听懂的小伙伴，关注点个赞，下次不迷路。

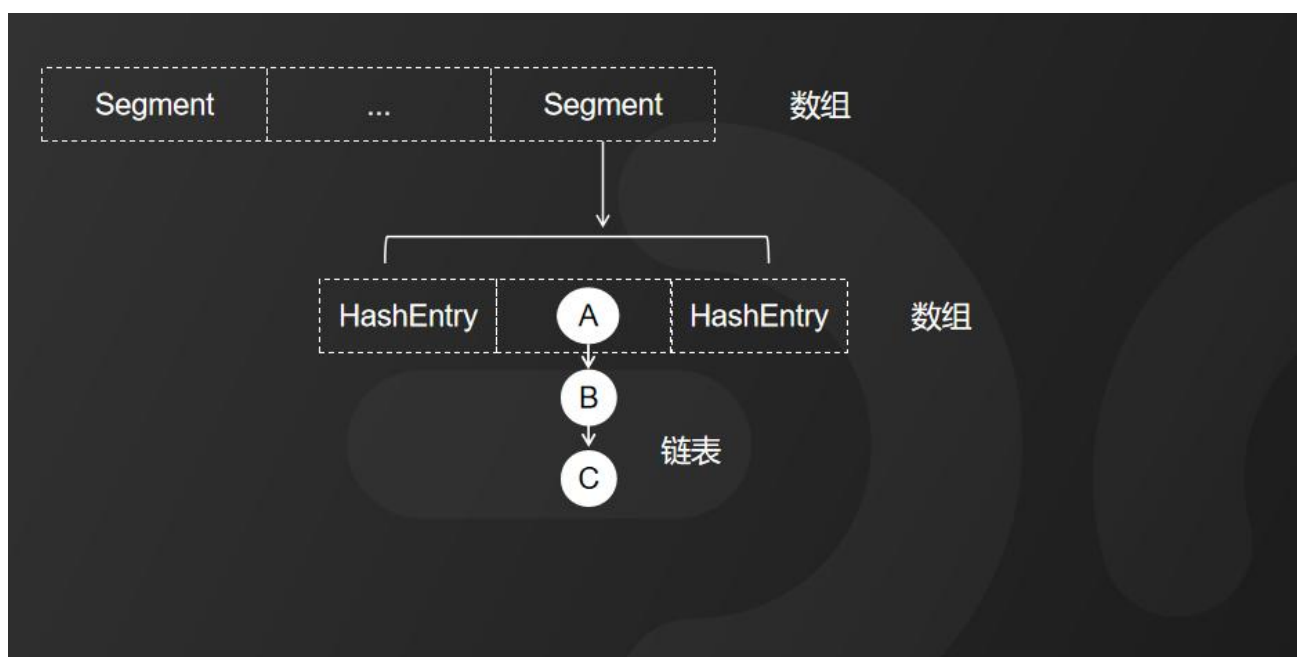
我是被编程耽误的文艺 Tom，如果还有其他疑问或者需要高清无码图的小伙伴，请在评论区留言。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！3 人未读

15. ConcurrentHashMap 是如何保证线程安全的?

ConcurrentHashMap 相当于是 HashMap 的多线程版本，它的功能本质上和 HashMap 没什么区别。因为 HashMap 在并发操作的时候会出现各种问题，比如死循环问题、数据覆盖等问题。而这些问题，只要使用 ConcurrentHashMap 就可以完美地解决。那问题来了，ConcurrentHashMap 它是如何保证线程安全的呢？

1、JDK1.7 实现原理

首先，我们来看 JDK 1.7 中 ConcurrentHashMap 的底层结构，它基本延续了 HashMap 的设计，采用的是数组加链表的形式。和 HashMap 不同的是，ConcurrentHashMap 中的数组设计分为大数组 Segment 和小数组 HashEntry，来着这张图。



大数组 Segment 可以理解为一个数据库，而每个数据库（Segment）中又有很多张表（HashEntry），每个 HashEntry 中又有很多条数据，这些数据是用链表连接的。了解了 ConcurrentHashMap 的基本结构设计，我们再来看它的线程安全实现，就比较简单了。

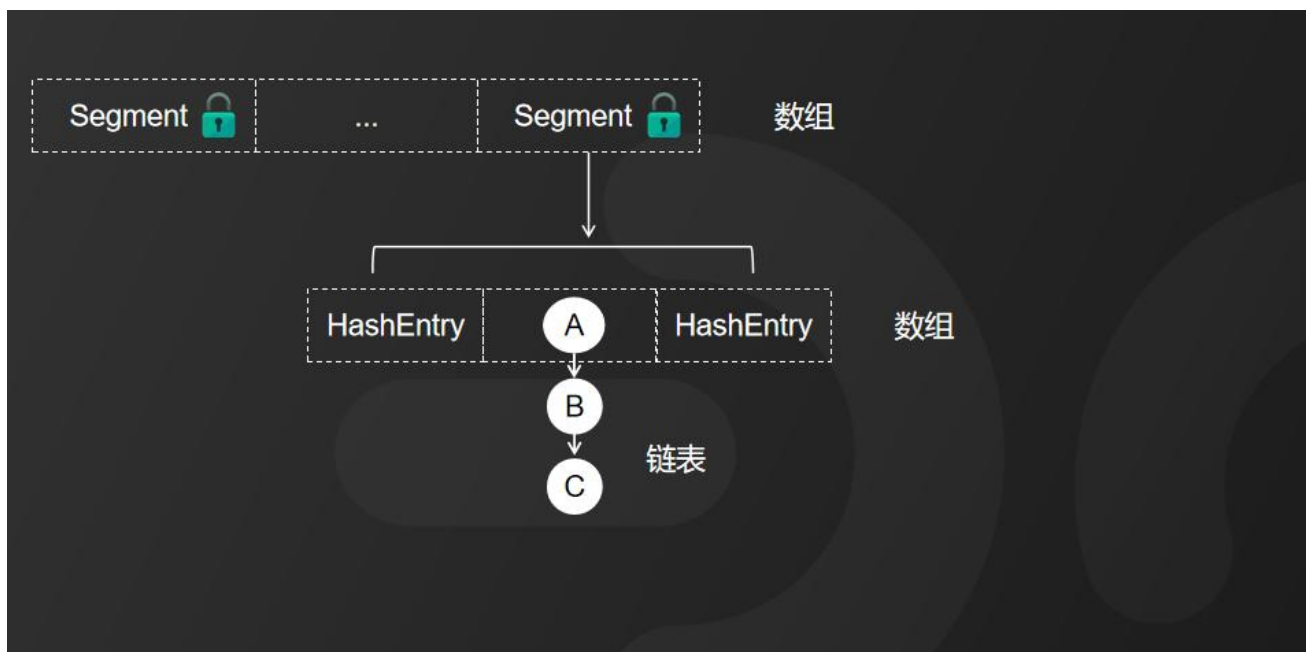
接下来我们来对照 JDK1.7 中 ConcurrentHashMap 的 put() 方法源码实现。

```

final V put(K key, int hash, V value, boolean onlyIfAbsent) {
    HashEntry<K,V> node = tryLock() ? null : scanAndLockForPut(key, hash, value);
    V oldValue;
    try {
        HashEntry<K,V>[] tab = table;
        int index = (tab.length - 1) & hash;
        HashEntry<K,V> first = entryAt(tab, index);
        for (HashEntry<K,V> e = first;;) {
            if (e != null) {
                K k;
                ...
            }
            else {
                ...
            }
        }
    } finally {
        unlock();
    }
    return oldValue;
}

```

因为 Segment 本身是基于 ReentrantLock 重入锁实现的加锁和释放锁的操作，这样就能保证多个线程同时访问 ConcurrentHashMap 时，同一时间只能有一个线程能够操作相应的节点，这样就保证了 ConcurrentHashMap 的线程安全。

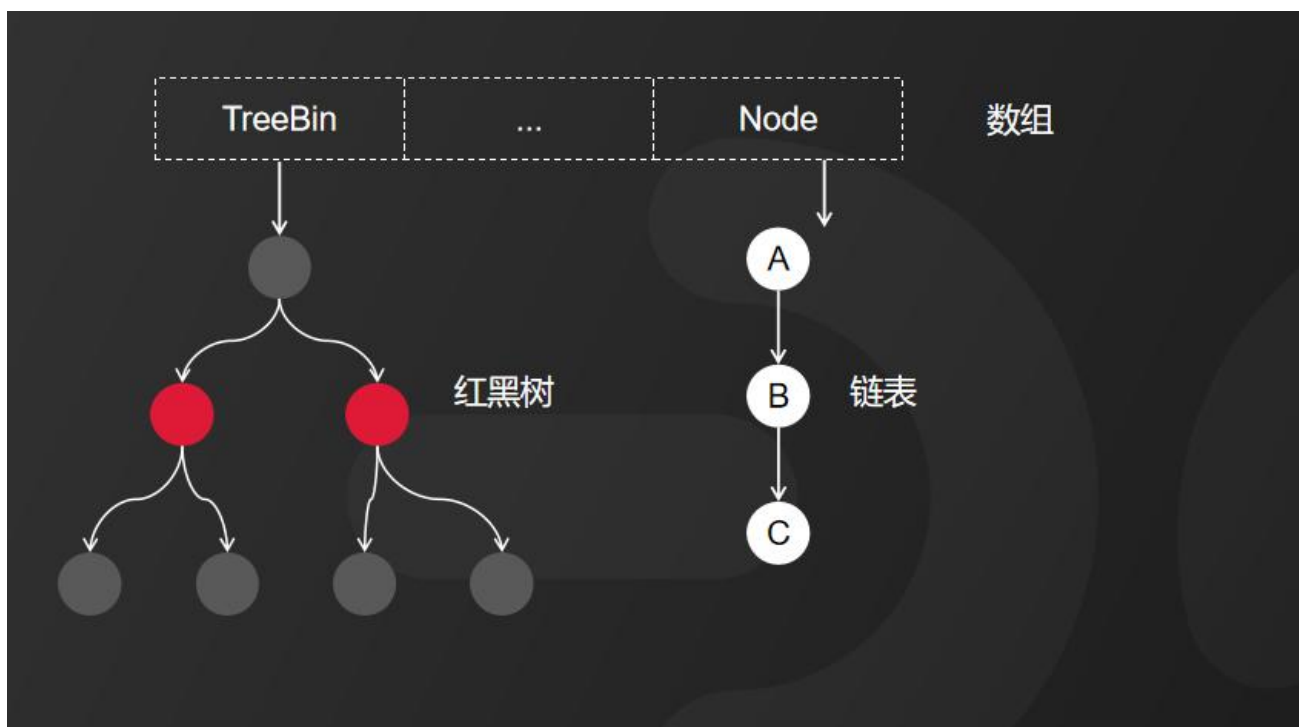


也就是说 ConcurrentHashMap 的线程安全是建立在 Segment 加锁的基础上的，所以，我们称它为分段锁或者片段锁，如图中所示。

那 JDK1.8 又是如何实现的呢？

2、JDK1.8 优化内容

在 JDK1.7 中，ConcurrentHashMap 虽然是线程安全的，但因为它的底层实现是数组加链表的形式，所以在数据比较多情况下，因为要遍历整个链表，会降低访问性能。所以，JDK1.8 以后采用了数组 加 链表 加 红黑树的方式优化了 ConcurrentHashMap 的实现，具体实现如图所示。



当链表长度大于 8，并且数组长度大于 64 时，链表就会升级为红黑树的结构。JDK 1.8 中的 ConcurrentHashMap 虽然保留了 Segment 的定义，但这，仅仅是为了保证序列化时的兼容性，不再有任何结构上的用处了。

那在 JDK 1.8 中 ConcurrentHashMap 的源码是如何实现的呢？它主要是使用了 CAS 加 volatile 或者 synchronized 的方式来保证线程安全。

```

final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode());
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            if (casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)))
                break;
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            synchronized (f) {
                ...
            }
            if (binCount != 0) {
                if (binCount >= TREEIFY_THRESHOLD)
                    treeifyBin(tab, i);
                if (oldVal != null)
                    return oldVal;
                break;
            }
        }
    }
    addCount(1L, binCount);
    return null;
}

```

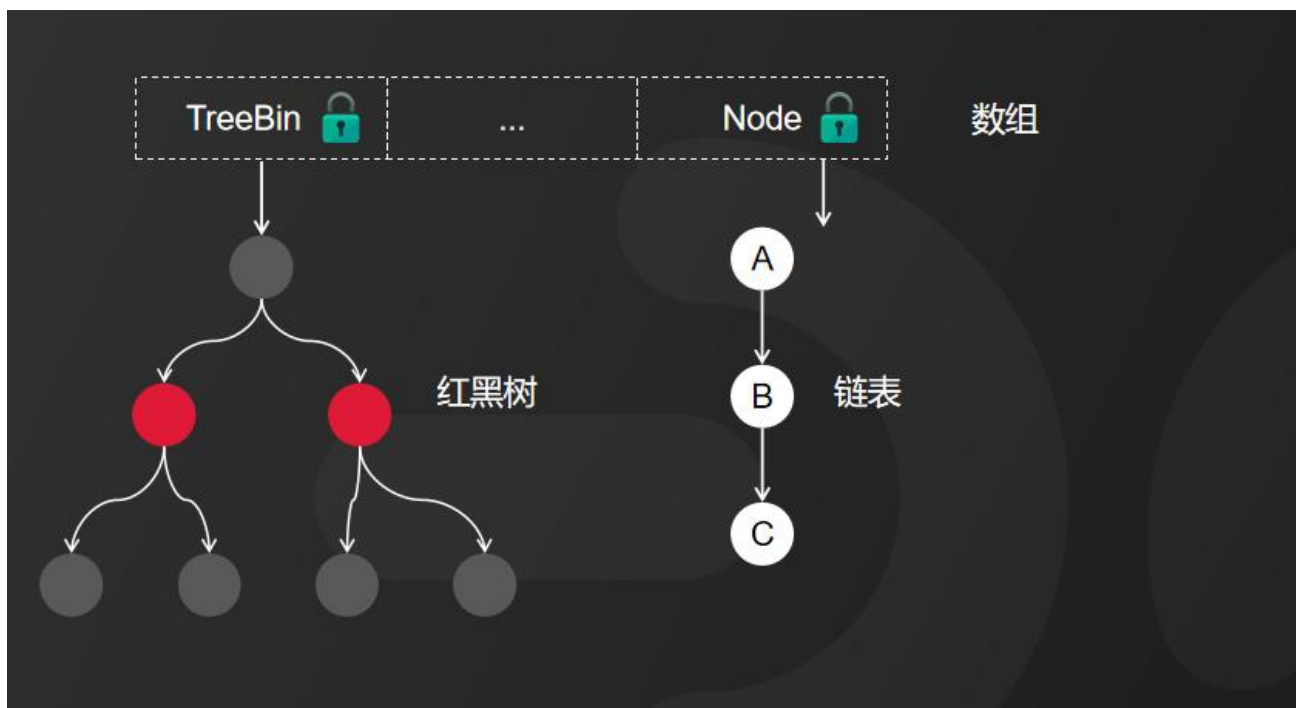
我们可以从源码片段中看到，添加元素时首先会判断容器是否为空，如果为空则使用 `volatile` 加 `CAS` 来初始化，如果容器不为空，则根据存储的元素计算该位置是否为空。

如果根据存储的元素计算结果为空则利用 `CAS` 设置该节点；

如果根据存储的元素计算为空不为空，则使用 `synchronized`，然后，遍历桶中的数据，并替换或新增节点到桶中，

最后再判断是否需要转为红黑树。这样就能保证并发访问时的线程安全了。

如果把上面的执行用一句话归纳的话，就相当于 `ConcurrentHashMap` 通过对头结点加锁来保证线程安全的。



这样设计的好处是，使得锁的粒度相比 Segment 来说更小了，发生 hash 冲突 和 加锁的频率也降低了，在并发场景下的操作性能也提高了。而且，当数据量比较大的时候，查询性能也得到了很大的提升。

2、总结

最后，我们来总结一下：



1、ConcurrentHashMap 在 JDK 1.7 中使用的数组 加 链表的结构，其中数组分为两类，大树组 Segment 和 小数组 HashEntry，而加锁是通过给 Segment 添加 ReentrantLock 重入锁来保证线程安全的。

2、ConcurrentHashMap 在 JDK1.8 中使用的是数组 加 链表 加 红黑树的方式实现，它是通过 CAS 或者 synchronized 来保证线程安全的，并且缩小了锁的粒度，查询性能也更高。

ConcurrentHashMap 中有很多设计思想是值得我们学习和借鉴的，比如说锁的粒度控制、分段锁的设计等等，都可以应用在实际的业务开发场景中。我们通过学习这些底层原理从中获取很多的设计思路，帮助我们更高效地去解决实际问题。

好了，关于 ConcurrentHashMap 的分享就讲到这里，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果还有其他疑问或者需要的小伙伴，请在评论区留言。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

16. 为什么 ConcurrentHashMap 不允许插入 null 值？

在 Java 语言中，给 ConcurrentHashMap 和 Hashtable 这些线程安全的集合中的 Key 或者 Value 插入 null（空）值的会报空指针异常，但是单线程操作的 HashMap 又允许 Key 或者 Value 插入 null（空）值。这到底是为什么呢？

1、探寻源码

为了找到原因，我们先来看这样一段源码片段，打开 ConcurrentHashMap 的 putVal() 方法，源码中第一句就非常明确地做了判断，如果 Key 或者 Value 为 null（空）值，就直接抛出空指针异常。

```
final V putVal(K key, V value, boolean onlyIfAbsent) {  
    if (key == null || value == null) throw new NullPointerException();  
    ...  
}
```

插入null值，在JDK源码中，直接抛出空指针异常

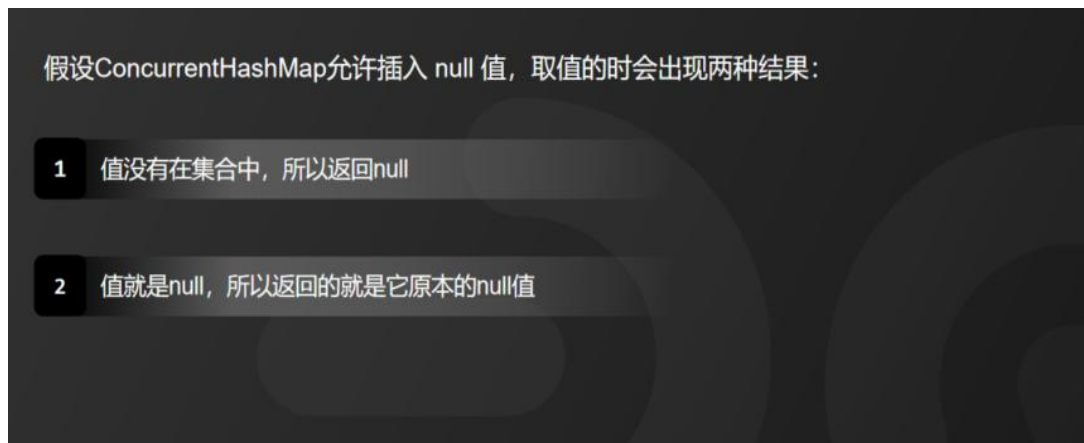
我们在源码中似乎已经找到了原因，你可以这样回答面试官，说 JDK 源码就是这么规定的。然而，这

个原因是不能说服面试官的，虽然，源码是这样设计的，我们要思考的是，这样设计背后更深层次的原因。

那到底为什么 `ConcurrentHashMap` 不允许插入 `null`（空）值，`HashMap` 又允许插入呢？

2、歧义问题

因为给 `ConcurrentHashMap` 中插入 `null`（空）值会存在歧义。我们可以假设 `ConcurrentHashMap` 允许插入 `null`（空）值，那么，我们取值的时候会出现两种结果：



1、值没有在集合中，所以返回的结果就是 `null`（空）；

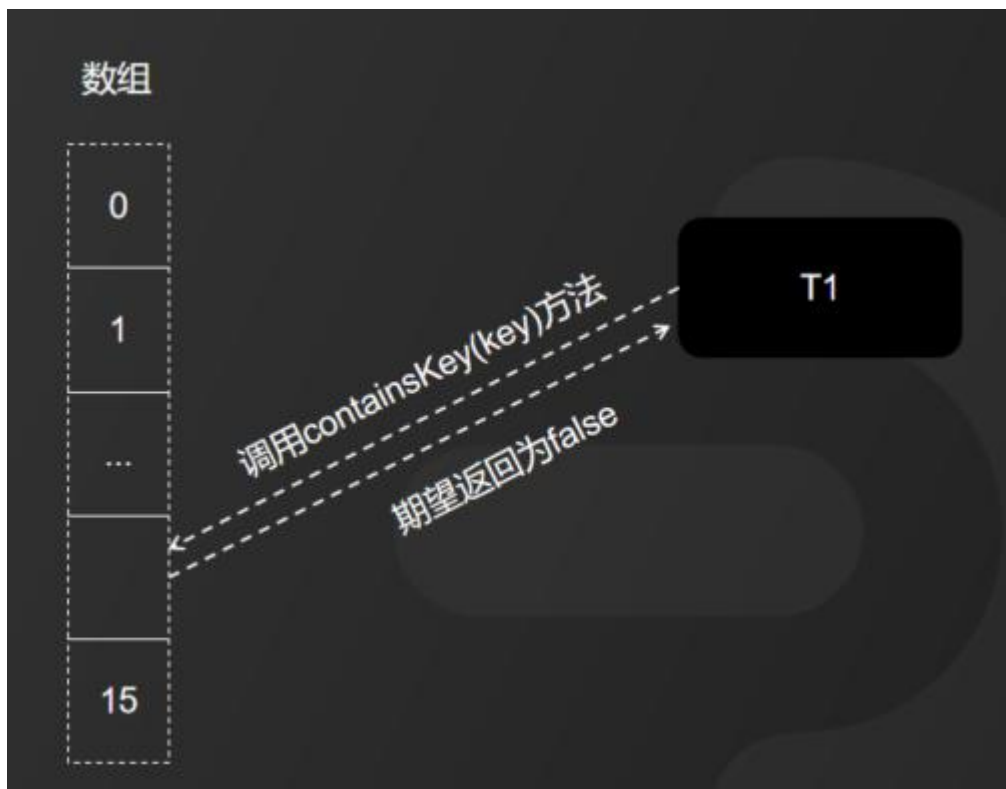
2、值就是 `null`（空），所以返回的结果就是它原本的 `null`（空）值。

这就产生了歧义问题。

那 `HashMap` 允许插入 `null`（空）值，难道它就不担心出现歧义吗？这是因为 `HashMap` 的设计是给单线程使用的，所以如果取到 `null`（空）值，我们可以通过 `HashMap` 的 `containsKey(key)` 方法来区分这个 `null`（空）值到底是插入值是 `null`（空），还是本就没有才返回的 `null`（空）值。

而 `ConcurrentHashMap` 就不一样了，因为 `ConcurrentHashMap` 是在多线程场景下使用的，它的情况更加复杂。

举个例子，现在有线程 `T1` 调用了 `ConcurrentHashMap` 的 `containsKey(key)` 方法，我们期望返回的结果是 `false`，也就是说，`T1` 并没有往 `ConcurrentHashMap` 中 `put null`（空）值。



但是，恰恰出了个意外，在线程T1还没有得到返回结果之前，线程T2又调用了ConcurrentHashMap的 put() 方法，插入了一个Key，并且存入的Value是 null（空）值。那么，线程T1 最终得到的返回结果就变成 true 了。



显然，这个结果和我们之前期望的 `false` 完全不一致。

也就是说，在多线程的复杂情况下，我们多线程的复杂情况下，到底是插入的 `null`（空）值，还是本就没有才返回的 `null`（空）值。也就是说，产生的歧义不能被证伪，

3、作者回复

对于 `ConcurrentHashMap` 不允许插入 `null` 值的问题，有人问过 `ConcurrentHashMap` 的作者 Doug Lea，以下是他回复的邮件内容：

The main reason that nulls aren't allowed in ConcurrentMaps (ConcurrentHashMaps, ConcurrentSkipListMaps) is that ambiguities that may be just barely tolerable in non-concurrent maps can't be accommodated. The main one is that if `map.get(key)` returns `null`, you can't detect whether the key explicitly maps to `null` vs the key isn't mapped.

In a non-concurrent map, you can check this via `map.containsKey(key)`, but in a concurrent one, the map might have changed between calls.

Further digressing: I personally think that allowing
nulls in Maps (also Sets) is an open invitation for programs
to contain errors that remain undetected until
they break at just the wrong time. (Whether to allow nulls even
in non-concurrent Maps/Sets is one of the few design issues surrounding
Collections that Josh Bloch and I have long disagreed about.)

It is very difficult to check for null keys and values
in my entire application .
Would it be easier to declare somewhere
static final Object NULL = new Object();
and replace all use of nulls in uses of maps with NULL?

-Doug

以上信件的主要意思是，Doug Lea 认为这样设计最主要的原因是：不容忍在并发场景下出现歧义！

4、总结

ConcurrentHashMap 在源码中加入不允许插入 null （空） 值的设计，主要目的是为了防止并发场景下的歧义问题。

以上就是我对关于 ConcurrentHashMap 为什么不允许插入 null （空） 值的解答，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果还有其他疑问或者需要高清无码图的小伙伴，可以关注我的主页加 V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

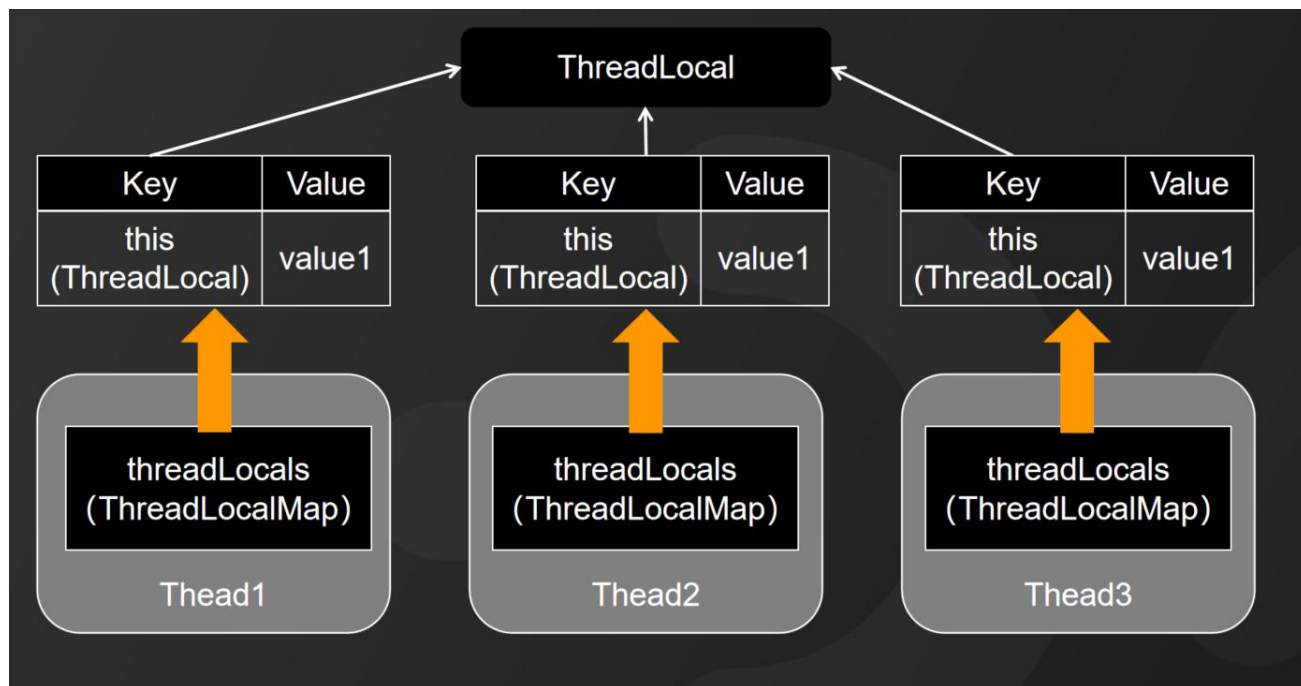
17. ThreadLocal 真的会造成内存泄漏吗？

ThreadLocal 在并发场景中，应用非常多。前几天有位小伙伴问我一个问题，说 ThreadLocal 是不是真的会造成内存泄漏？今天给大家做一个分享，个人见解，仅供参考。如果大家有其他见解可以在评论区讨论。

1、ThreadLocal 的基本原理

考虑到很多小伙伴可能还不太了解 ThreadLocal，我先简单介绍一下 ThreadLocal。在多线程并发访问同一个共享变量的情况下，如果不做同步控制的话，就可能会导致数据不一致的问题，所以，我们需要使用 synchronized 加锁来解决。

而 ThreadLocal 换了一个思路来处理多线程的情况，



`ThreadLocal` 本身并不存储数据，它使用了线程中的 `threadLocals` 属性，`threadLocals` 的类型就是在 `ThreadLocal` 中的定义的 `ThreadLocalMap` 对象，当调用 `ThreadLocal` 的 `set(T value)` 方法时，`ThreadLocal` 将自身的引用也就是 `this` 作为 `Key`，然后，把用户传入的值作为 `Value` 存储到线程的 `ThreadLocalMap` 中，这就相当于每个线程的读写操作都是基于线程自身的一个私有副本，线程之间的数据是相互隔离的，互不影响。

这样一来基于 `ThreadLocal` 的操作也就不存在线程安全问题了。它相当于采用了用空间来换时间的思路，从而提高程序的执行效率。

2、四种对象引用

在 `ThreadLocalMap` 内部，维护了一个 `Entry` 数组 `table` 的属性，用来存储键值对的映射关系，来看这样一段代码片段：

```

static class ThreadLocalMap {
    ...
    private Entry[] table;

    static class Entry implements WeakReference<ThreadLocal<?>> {
        Object value;
        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
    ...
}

```

```

static class ThreadLocalMap {

    ...

    private Entry[] table;

    static class Entry implements WeakReference<ThreadLocal<?>> {

        Object value;

        Entry(ThreadLocal<?> k, Object v) {

            super(k);

            value = v;

        }

    }

    ...

}

```

Entry 将 ThreadLocal 作为 Key，值作为 Value 保存，它继承自 WeakReference，注意构造函数

里的第一行代码 `super(k)`，这意味着 `ThreadLocal` 对象是一个「弱引用」。有的小伙伴可能对「弱引用」不太熟悉，这里再介绍一下 Java 的四种引用关系。

在 JDK1.2 之后，Java 对引用的概念做了一些扩充，将引用分为“强”、“软”、“弱”、“虚”四种，由强到弱依次为：



强引用：指代码中普遍存在的赋值行为，如：`Object o = new Object()`，只要强引用关系还在，对象就永远不会被回收。

软引用：还有用处，但不是必须存活的对象，JVM 会在内存溢出前对其进行回收，例如：缓存。

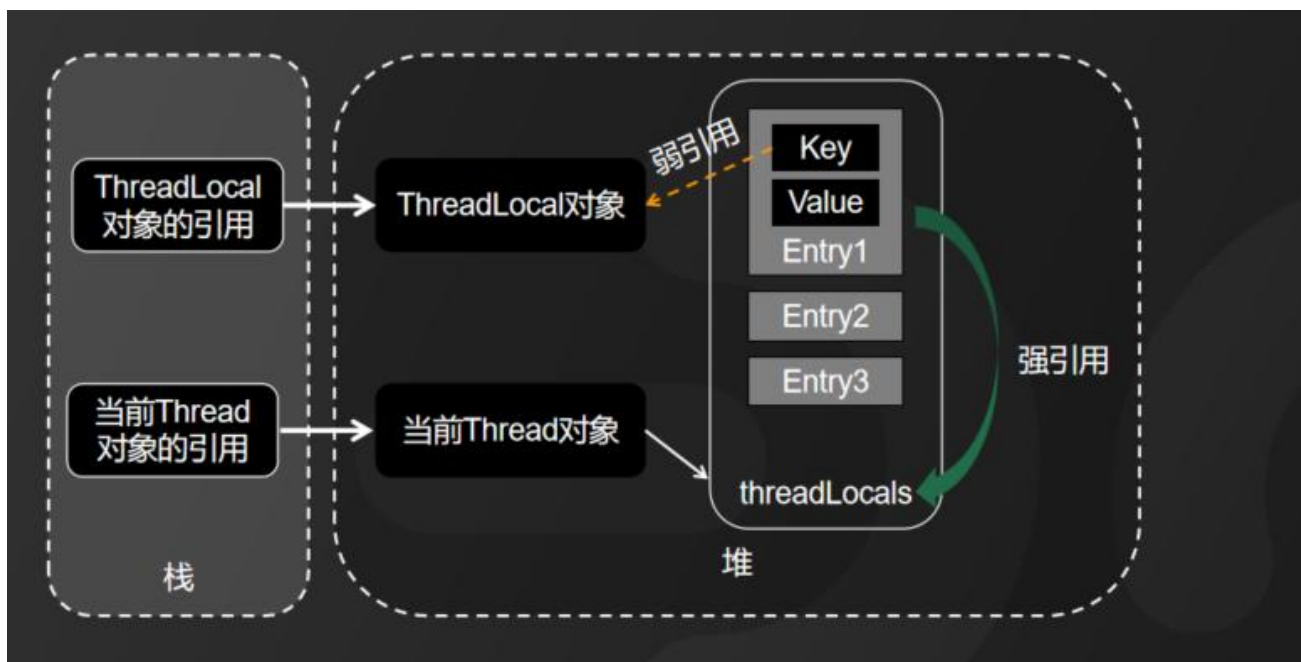
弱引用：非必须存活的对象，引用关系比软引用还弱，不管内存是否够用，下次 GC 一定回收。

虚引用：也称“幽灵引用”、“幻影引用”，最弱的引用关系，完全不影响对象的回收，等同于没有引用，虚引用的唯一的目的是对象被回收时会收到一个系统通知。

这个描述还是比较官方的，简单总结一下，大家应该都追过剧，强引用就好比是男主角，怎么都死不了。软引用就像女主角，虽有一段经历，还是没走到最后。弱引用就是男二号，注定用来牺牲的。虚引用就是路人甲了。

3、造成内存泄漏的原因

内存泄漏和 `ThreadLocalMap` 中定义的 `Entry` 类有非常大的关系。



这个动画完整地展示了 ThreadLocal 中对象引用的关系，需要这张高清图的小伙伴可以在评论区留言。

由于 ThreadLocal 对象是弱引用，如果外部没有强引用指向它，它就会被 GC 回收，导致 Entry 的 Key 为空 (null)，如果这时 Value 外部也没有强引用指向它，那么 Value 就永远也访问不到了，按理也应该被 GC 回收，但是由于 Entry 对象还在强引用 Value，导致 Value 无法被回收，这时「内存泄漏」就发生了，Value 成了一个永远也无法被访问，但是又无法被回收的对象。

Entry 对象属于 ThreadLocalMap，ThreadLocalMap 又属于 Thread，如果线程本身的生命周期很短，短时间内就会被销毁，那么「内存泄漏」立刻就会得到解决，只要线程被销毁，Value 也会随之被回收。

问题是，线程本身是非常珍贵的计算机资源，很少会去频繁的创建和销毁，一般都是通过线程池来使用，这就将线程的生命周期大大拉长，「内存泄漏」的影响也会越来越大。

最后，一句话总结一下。

造成内存泄漏的原因总结：

threadLocals对象中的Entry对象不再使用后，如果没有及时清除Entry对象，而程序自身也无法通过垃圾回收机制自动清除，就可能导致内存泄漏。

threadLocals 对象中的 Entry 对象不再使用后，如果没有及时清除 Entry 对象，而程序自身也无法通过垃圾回收机制自动清除，就可能导致内存泄漏。

4、如何避免内存泄漏？

不要听到「内存泄漏」就不敢使用 ThreadLocal, 只要规范化使用是不会有问题的。我给大家支几个招：

如何避免内存泄漏

- 1 每次使用完ThreadLocal都记得调用remove()方法清除数据
- 2 将ThreadLocal变量尽可能地定义成static final

1、每次使用完 ThreadLocal 都记得调用 remove() 方法清除数据。

2、将 ThreadLocal 变量尽可能地定义成 static final，避免频繁创建 ThreadLocal 实例。这样也就保证程序一直存在 ThreadLocal 的强引用，也能保证任何时候都能通过 ThreadLocal 的弱引用访问到 Entry 的 Value 值，进而清除掉。

当然，就是使用不规范，ThreadLocal 内部也做了一些优化，比如：



1、调用 `set()` 方法时，ThreadLocal 会进行采样清理、全量清理，扩容时还会继续检查。

2、调用 `get()` 方法时，如果没有直接命中或者向后环形查找时也会进行清理。

3、调用 `remove()` 时，除了清理当前 Entry，还会向后继续清理。

所以，我真的希望小伙伴们，不要被制造焦虑的面试官们卷得太深。听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加 V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

18. Spring Boot 自动装配原理

昨天，有位在广州工作 4 年的小伙伴，在面试中被问到 SpringBoot 自动装配原理，当时，自我感觉比较好，他要的是 30K，但是都没有拿到 Offer。今天，我给大家分享一下我的理解。

1、Spring Boot 自动装配是什么？

SpringBoot 自动装配主要是基于注解编程 和 约定优于配置的思想来设计的。

自动装配：就是由Spring自动把其他组件中的Bean装载到IoC容器中，不需要开发人员再去配置文件中添加大量的配置。

```
@SpringBootApplication
public class QuickStartApplication {
    public static void main(String[] args){
        SpringApplication.run(QuickStartApplication.class,args);
    }
}
```

Spring 和 Spring Boot的最大区别就是在于Spring Boot的自动装配。

自动装配就是由 Spring 自动把其他组件中的 Bean 装载到 IoC 容器中，不需要开发人员再去配置文件中添加大量的配置。我们只需要在 Spring Boot 的启动类上添加 @SpringBootApplication 注解，开启自动装配。

这种自动装配的思想，在 Spring 3.x 以后就开始支持，我们只要在类上添加 @Enable 注解就可以了，只是没有像 Spring Boot 这样全面地去设计。

因此，Spring 和 Spring Boot 的最大区别就是在于 Spring Boot 的自动装配。那自动装配的原理又是什么呢？

2、自动装配原理

@SpringBootApplication 这个注解是暴露给用户使用的入口，它的底层是由 @EnableAutoConfiguration 这个注解来实现的。

这样一个自动装配的实现，我把它归纳为以下三个核心步骤：

自动装配的实现，归纳为以下三个核心步骤：

- 1 组件中必须要包含 `@Configuration` 的配置类
- 2 第三方jar包，在/META-INF/目录下增加 `spring.factories` 文件
- 3 Spring调用 `ImportSelector` 接口完成动态加载

第一步：启动依赖组件的时候，组件中必须要包含 `@Configuration` 的配置类，在这个配置类里面声明为 `@Bean` 注解，就将方法的返回值或者属性值 注入到 IoC 容器中。

第二步：如果是使用第三方 jar 包，Spring Boot 采用 SPI 机制，只需要在/META-INF/目录下增加 `spring.factories` 配置文件。然后，Spring Boot 会根据约定规则，自动使用 `SpringFactoriesLoader` 来加载配置文件中的内容。

第三步：Spring 获取到第三方 jar 中的配置以后，会使用调用 `ImportSelector` 接口来完成动态加载。

这样的设计的好处在于，大幅减少了臃肿的配置文件，而且各模块之间的依赖实现了深度解耦。比如，我们使用 Spring 创建 Web 程序时需要导入非常多的 Maven 依赖，而 Spring Boot 只需要一个 Maven 依赖来创建 Web 程序，并且 Spring Boot 还把我们最常用的依赖都放到了一起，我们只需要 引入 `spring-boot-starter-web` 这一个依赖就可以完成一个简单的 Web 应用。

以前用 Spring 的时候需要 XML 文件配置开启一些功能，现在 Spring Boot 不用 XML 配置了，只需要写一个，加了 `@Configuration` 注解 或者实现 对应接口的配置类就可以了。

3、总结

最后，我总结一下，Spring Boot 自动装配是 Spring 的完善和扩展，就是为我们便捷开发，方便测试和部署，提高效率而诞生的框架技术。

小伙伴们，如果你被问到过 Spring Boot 自动装配原理的问题，你是怎么回答的呢？可以在评论区分享你的回答。

我是被编程耽误的文艺 Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加 V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再

难！

19. 哪些情况下的单例对象可能会破坏？

1、单例模式的定义

关于单例模式的定义，官方原文是这样描述的：

官方原文：

Ensure a class has only one instance, and provide a global point of access to it.

大致意思是，确保一个类在任何情况下都绝对只有一个实例，并提供一个全局访问点。

Ensure a class has only one instance, and provide a global point of access to it.

大致意思是，确保一个类在任何情况下都绝对只有一个实例，并提供一个全局访问点。

单例模式的写法相信只要是程序员应该都会，也很非常简单，这里我就不一一列举了。今天，我要重点要给大家分析的是，在 Java 中，哪些单例对象是最有可能被破坏的。

2、单例被破坏的五个场景



我把可能出现单例被破坏的情况，一共归纳为五种，分别为多线程破坏单例、指令重排破坏单例、克隆破坏单例、反序列化破坏单例、反射破坏单例。

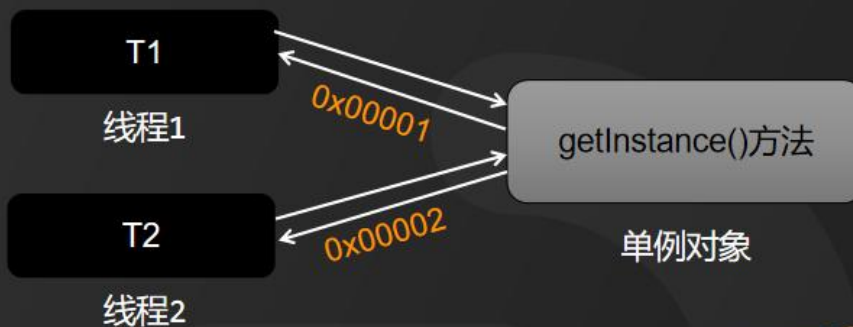
下面我详细分析一下每种情况并给出解决方案：

第一种：多线程破坏单例

1

多线程破坏单例

问题： 多个线程同时操作，导致同时创建多个对象



解决方案：

- 1 改为DCL双重检查锁的写法
- 2 使用静态内部类的写法，性能更高

在多线程环境下，线程的时间片是由 CPU 自由分配的，具有随机性，而单例对象作为共享资源可能会同时被多个线程同时操作，从而导致同时创建多个对象。当然，这种情况只出现在懒汉式单例中。如果是饿汉式单例，在线程启动前就被初始化了，不存在线程再创建对象的情况。

如果懒汉式单例出现多线程破坏的情况，我给出以下两种解决方案：

- 1、改为 DCL 双重检查锁的写法。
- 2、使用静态内部类的写法，性能更高。

第二种：指令重排破坏单例

2

指令重排破坏单例

问题： JVM指令重排可能导致懒汉式单例被破坏

```
instance = new Singleton()
```

执行指令：

```
memory = allocate()    ctorInstance(memory)    instance = memory
```

1、分配内存

2、初始化对象

3、赋值引用

指令重排：

```
memory = allocate()    ctorInstance(memory)    instance
```

1、分配内存

2、赋值引用

3、初始化对象

解决方案：

```
1 private static volatile Singleton instance = null;
```

指令重排也可能导致懒汉式单例被破坏。来看这样一句代码：

```
instance = new Singleton();
```

看似简单的一段赋值语句：`instance = new Singleton();`

其实 JVM 内部已经被转换为多条执行指令：

```
memory = allocate();    分配对象的内存空间指令  
ctorInstance(memory);   初始化对象  
instance = memory;      将已分配存地址赋值给对象引用
```

- 1、分配对象的内存空间指令，调用 `allocate()` 方法分配内存。
- 2、调用 `ctorInstance()` 方法初始化对象
- 3、将已分配存地址赋值给对象引用

但是经过重排序后，执行顺序可能是这样的：

```
memory = allocate();    分配对象的内存空间指令  
instance = memory;      将已分配存地址赋值给对象引用  
ctorInstance(memory);   初始化对象
```

- 1、分配对象的内存空间指令
- 2、设置 `instance` 指向刚分配的内存地址
- 3、初始化对象

我们可以看到指令重排之后，instance 指向分配好的内存放在了前面，而这段内存的初始化的指令被排在了后面，在线程 T1 初始化完成这段内存之前，线程 T2 虽然进不去同步代码块，但是在同步代码块之前的判断就会发现 instance 不为空，此时线程 T2 获得 instance 对象，如果直接使用就可能发生错误。

如果出现这种情况，我该如何解决呢？只需要在成员变量前加 volatile，保证所有线程的可见性就可以了。

```
private static volatile Singleton instance = null;
```

第三种：克隆破坏单例



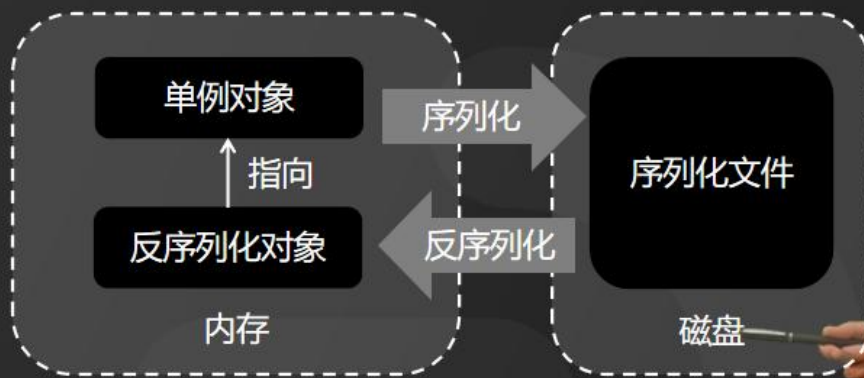
在 Java 中，所有的类就继承自 Object，也就是说所有的类都实现了 clone() 方法。如果是深 clone()，每次都会重新创建新的实例。那如果我们定义的是单例对象，岂不是也可调用 clone() 方法来反复创建新的实例呢？确实，这种情况是有可能发生的。为了避免发生这样结果，我们可以在单例对象中重写 clone() 方法，将单例自身的引用作为返回值。这样，就能避免这种情况发生。

第四种：反序列化破坏单例

4

反序列化破坏单例

问题：反序列化对象会重新分配内存，相当于重新创建对象



解决方案：

- 1 需要重写 `readResolve()` 方法，将返回值设置为单例对象

我们将 Java 对象序列化以后，对象通常会被持久化到磁盘或者数据库。如果我们要再次加载到内存，就需要将持久化的内容反序列化成 Java 对象。反序列化是基于字节码来操作的，我们要序列化以前的内容进行反序列化到内存，就需要重新分配内存，也就是说，要重新创建对象。那如果要反序列化的对象恰恰是单例对象，我们该怎么办呢？

我告诉大家一种解决方案，在反序列的过程中，Java API 会调用 `readResolve()` 方法，可以通过获取 `readResolve()` 方法的返回值覆盖反序列化创建的对象。

因此，只需要重写 `readResolve()` 方法，将返回值设置为已经存在的单例对象，就可以保证反序列化以后的对象是同一个了。之后再反序列化后的对象中的值，克隆到单例对象中。

第五种：反射破坏单例

5

反射破坏单例

问题： 反射可以任意调用私有构造方法创建单例对象



解决方案：

- 1 在构造方法中检查单例对象，如果已创建则抛出异常
- 2 将单例的实现方式改为枚举式单例

以上讲的所有单例情况都有可能被反射破坏。因为 Java 中的反射机制是可以拿到对象的私有的构造方法，也就是说，反射可以任意调用私有构造方法创建单例对象。当然，没有人会故意这样做，但是如果出现意外的情况，该如何处理呢？我推荐大家两种解决方案，

第一种方案是在所有的构造方法中第一行代码进行判断，检查单例对象是否已经被创建，如果已经被创建，则抛出异常。这样，构造方法将会被终止调用，也就无法创建新的实例。

第二种方案，将单例的实现方式改为枚举式单例，因为在 JDK 源码层面规定了，不允许反射访问枚

五、Java 开发程序员职业路线图



